

Lab 3: Hardware-based True Random Number Generator and Timer Control on
FPGA

Victor de Oliveira Venancio

ECE5440

ID: 9048

1.Introduction

To start the game, the player need to authenticate themselves by entering the last four digits of the creator's Peoplesoft ID using the four rightmost switches on the board. Each digit is set by toggling the four switches to high and low combinations so it matches the binary form of the corresponding digit. To enter that digit, they push the third button from left in the board, the password button (Figure 1). This is done four times for each digit. If the authentication is successful, LED 4 (the rightmost) will be turned on, otherwise, LED 3 (the second rightmost) will remain on indicating that the players are still logged out and need to try to authenticate again.

If the player is successfully authenticated a timer will output the number 99 in the two leftmost displays, indicating that the player will have a total of 99 seconds to play.

To start the game the player presses the third button from left in the board again (which in this case works as the game start button - Figure 1). The time will begin decreasing by one each second and the player must play as many rounds as they can before the timer indicates 00.

On each round, the player will press the Random Number Generation Button, which is the leftmost button on the FPGA – Figure 1). It will display a random hexadecimal number on the second rightmost display (Random Number Display) that varies depending on how long the player presses the button. Then player enters a binary number using the four rightmost switches and presses the second button to the left (Load Button – Figure 1) trying to enter a number whose sum with the random generated number is 15 in decimals, or f in hexadecimal. The number entered by the Load Button will be displayed in hexadecimal form on the rightmost display (Player Display – Figure 1) and the sum of their numbers will be displayed on the third rightmost display (Sum Display – Figure 1). If the sum shown in that display is F, then the leftmost LED in the FPGA board will be turned on indicating that the sum is 15 and the player who entered the second number gets one point in the total score from that round. If the sum is not F, then the second leftmost LED is turned on instead and no one receives a point in that round. The process repeats itself until time shows 00. When this happens, the game is over and the player's total score, counting all the times the player successfully matched the 15-sum, is shown in the two displays at the center (Score Displays – Figure 1). The player cannot enter any other number. If they want to play again they can press the third button from left (Restart Button – Figure 1) setting the timer back to 99 seconds, then press the same button again for the new game to start. The rightmost button (Reset Button) will reset the system when pressed, automatically logging player out. The rightmost LED is off and the second rightmost LED is on.

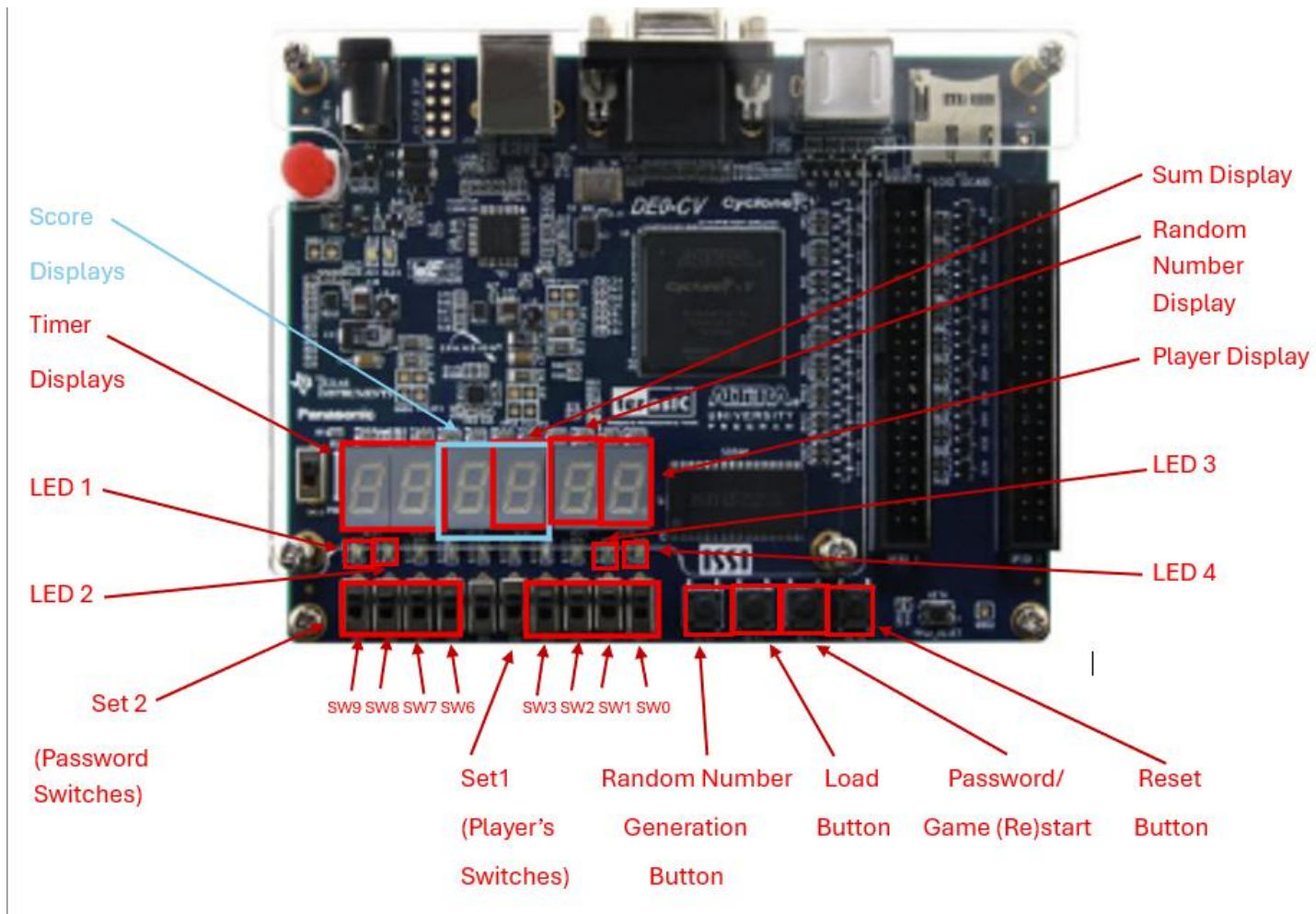


Figure 1. FPGA Board showing the player's switches, buttons, the displays and the LEDs.

Set 1 defines the switches used by Player 1 during the game:

SW3 – bit 3

SW2 – bit 2

SW1 – bit 1

SW0 – bit 0

Set 2 defines the switches used for authentication (log in):

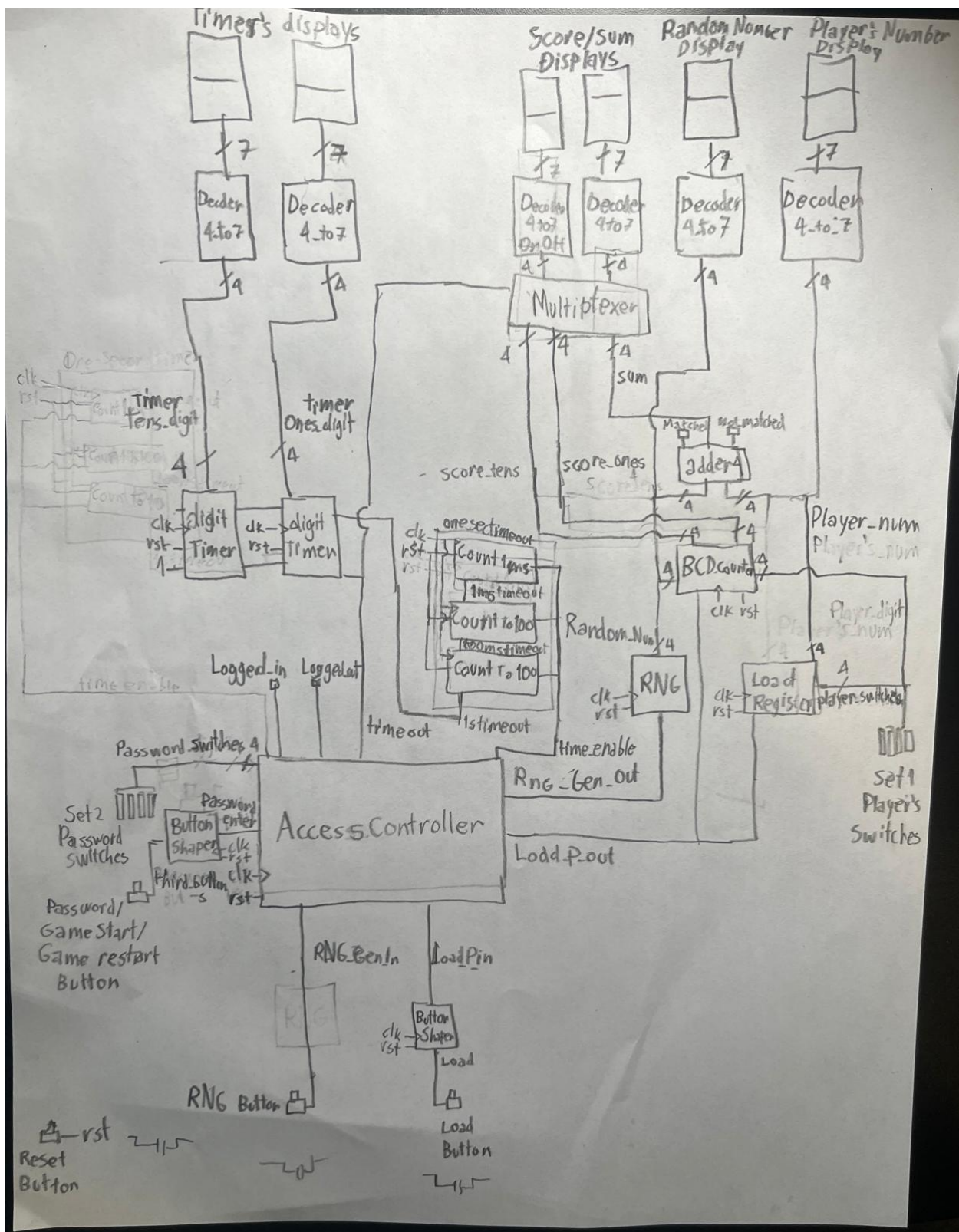
SW9 – bit 3

SW8 – bit 2

SW7 – bit 1

SW6 – bit 0

2. System Architecture Design:

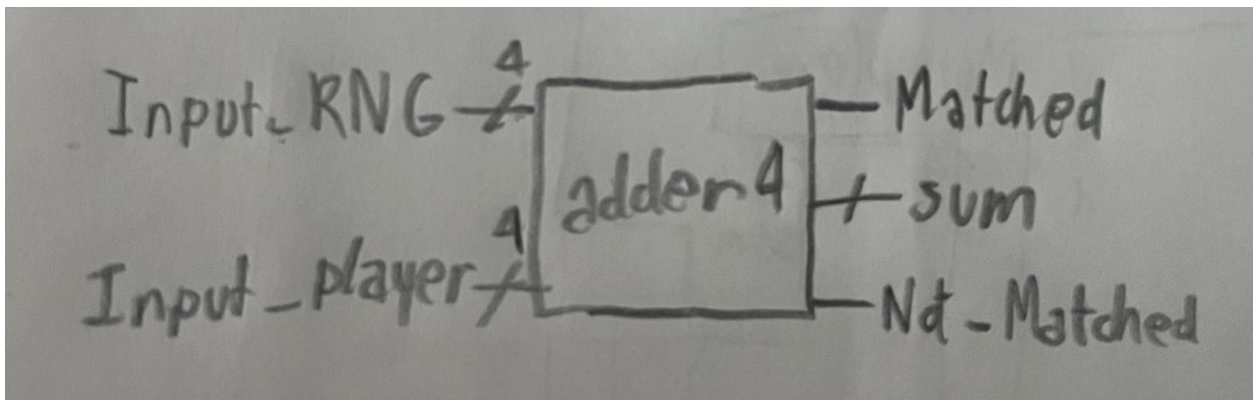


Matched is the bit corresponding to the case when sum4 equals 1111_2 or f_{16} . It will turn on the leftmost LED on the FPGA and indicate the targeted number of the sum was achieved

Not_matched is the bit corresponding to the case when sum4 does not 1111_2 or f_{16} . It will turn on the rightmost LED on the FPGA and indicate the targeted number of the sum was not achieved

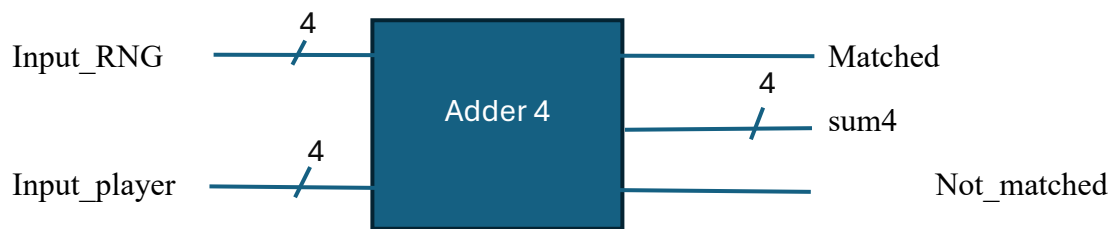
Modules:

- **Adder4**

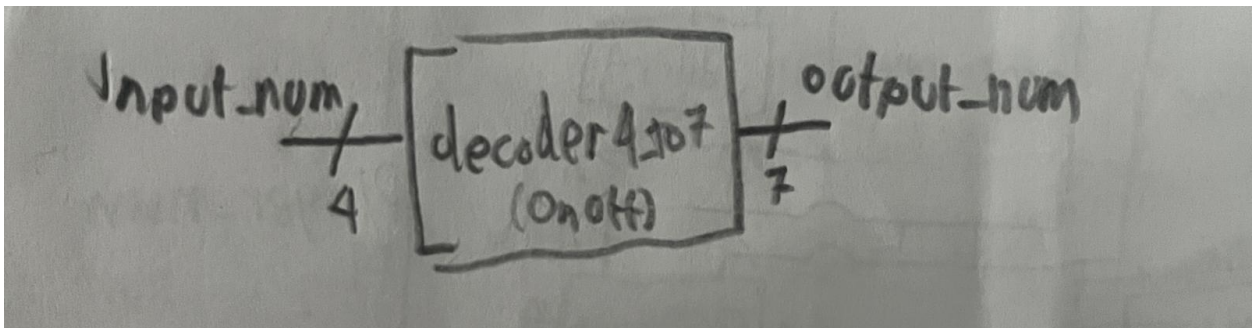


- Description: Adder 4 is the model that inputs the two 4-bit binary numbers. One number, input_RNG is the 4-bit binary random number output by the Random Number Generator module. The other number, input_player is the 4-bit binary number output by the Load Register module attached to the Load button, matching the high-low combination of 4 toggle switches in Set 1 (Figure 1). The module computes their sum (sum4) based on binary addition. Besides, it takes the sum output and see if it equals 1111_2 or f_{16} to determine if the LED corresponding to the sum equaling that value should be on (the signal Matched becomes high and Not-matched becomes low) or the LED corresponding to the otherwise case (signal Matched becomes low and Not-matched becomes high).
- Signal Definition:
 - input_RNG (signal_len: 4 bits) is the 4-bit binary number input by the random number generator.
 - input_player (signal_len: 4 bits) is the 4-bit binary number input by the load register.
 - sum4 (signal_len: 4 bits) is the 4-bit binary number output by adder4 resulting from the binary sum of input_RNG and input_player that will be wired to a Decode_4to7 to decode it into the number displayed in the Sum Display on FPGA (Figure 1).

- Matched (signal_len: 1 bit) is the signal that indicates that the sum of input_RNG and input_player is 15, being high when $\text{sum4} = 4'b1111$ and low otherwise.
- Not_matched (signal_len: 1 bit) is the signal that indicates that the sum of input_RNG and input_player is 15, being low when $\text{sum4} = 4'b1111$ and high otherwise.
-

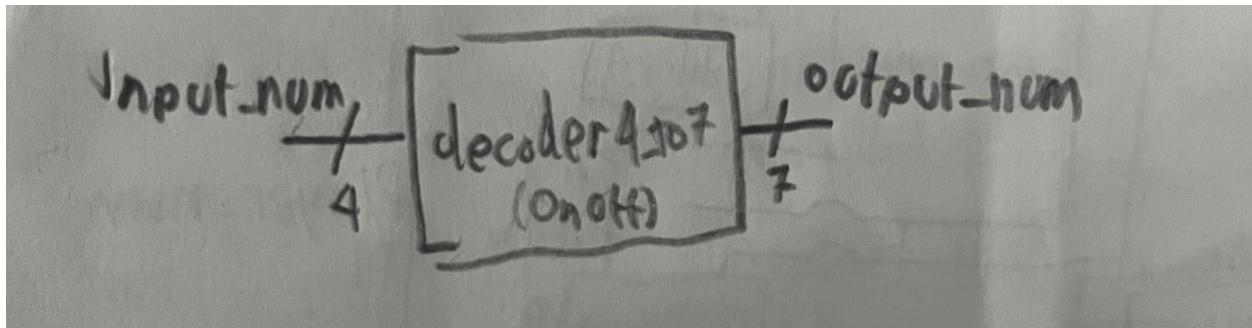


- **Decode_4to7:**



- Description: Decode_4to7 takes a 4-bit binary number (input_num) and decodes it into the 7 bits that will output the hexadecimal equivalent (output_player1) to that number into a displayer.
- Signal Definition:
 - input_num (signal_len: 4 bits) is the 4-bit binary number input
 - output_num (signal_len: 7 bits) is the 7-bit number decoded by Decode_4to7_1 from input_num to turn on the corresponding segments of its attached displayer to show the input binary number in hexadecimal form

- **Decode_4to7OnOff:**

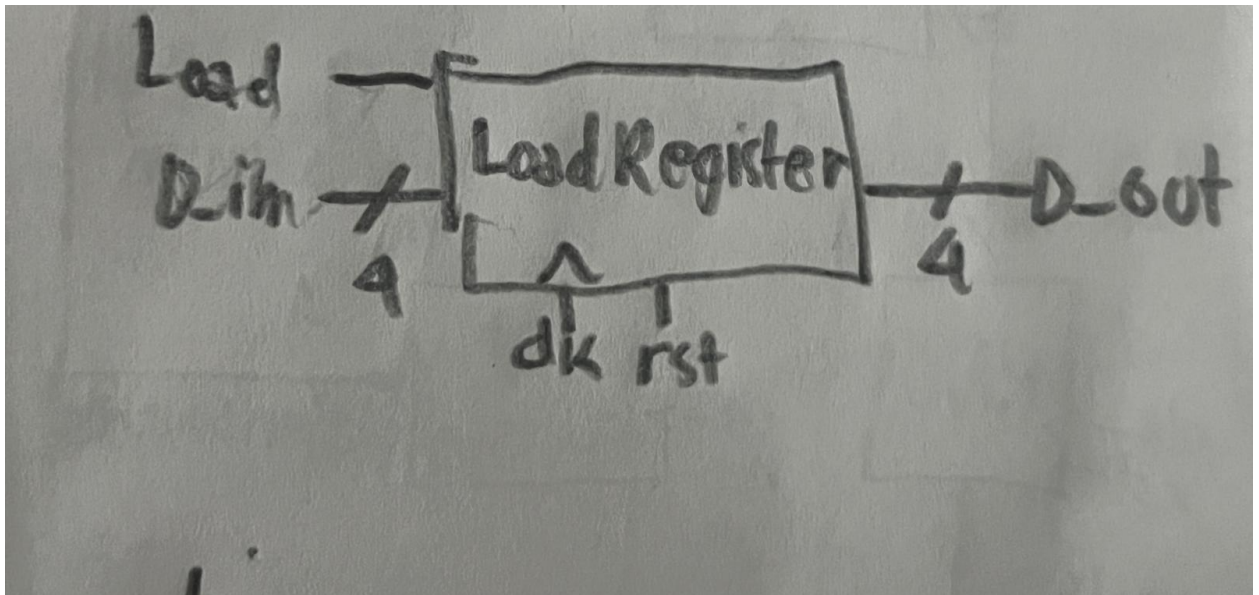


- Description: Decode_4to7 takes a 4-bit binary number (input_num) and decodes it into the 7 bits that will output the decimal equivalent (output_player1) to that number into a displayer. If Input_num is greater than 9, output_num will be 7'b1111111 essentially turning the display off.
- Signal Definition:
 - input_num (signal_len: 4 bits) is the 4-bit binary number input
 - output_num (signal_len: 7 bits) is the 7-bit number decoded by Decode_4to7_1 from input_num to turn on the corresponding segments of its attached displayer to show the input binary number in decimal form. If Input_num is greater than 9, output_num will be 7'b1111111 essentially turning the display off.

4
—/—

7
—/—

- **Load Register:**

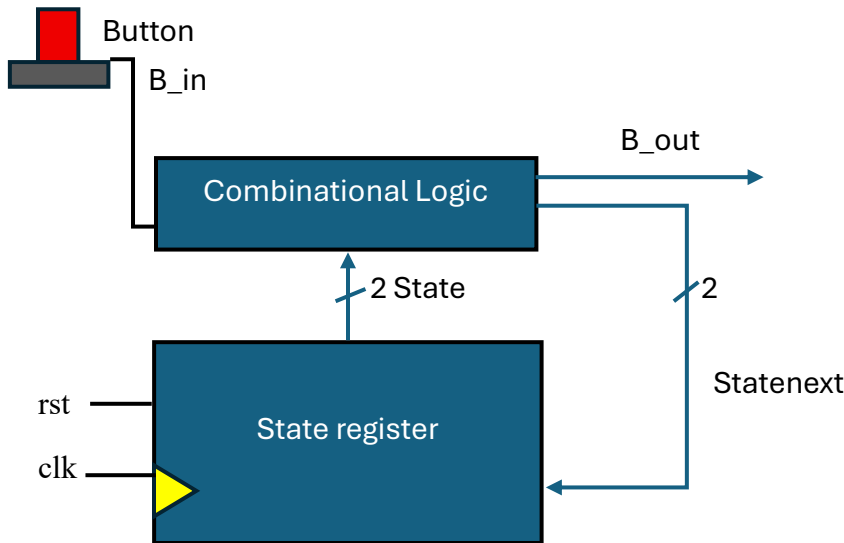


- Description: Load Register takes the 4-bit binary number entered by the user (D_in) and a pulse (Load) that is transmitted from the button shaper when that user presses their load button. Before the pulse is input in the Load Register, it passes through the access controller, which will only send the signal to the load register when the game is on (player logged in and pressed the Game Start button – Figure 1). If the reset button is not pressed, the Load Register outputs the same number entered by D_in in a 4-bit signal called D_out to a decoder to display the player's number and the adder for a sum. Otherwise D_out will be zero (4'b0000).
- Signal Definition:
 - D_in (signal_len: 4 bits) is the 4-bit binary number input by the player.
 - clk (signal_len: 1 bit) is the system clock.
 - rst (signal_len: 1 bit) is the reset signal that is high whenever the reset button is pressed.
 - Load (signal_len: 1 bit) is the signal that will allow or not the load register to output the number entered by the user to be decoded for a display and to the adder for sum. It will only input a high pulse signal when timer is running, allowing D_in's value to be output to the next module by D_out. Otherwise, Load will remain low and D_out will be the same it was before.
 - D_out (signal_len: 4 bits) is the 4-bit binary output of the load register. When Load is high, D_out will be the same as D_in and will be the same as before otherwise, keeping display unchanged.

- **Button Shaper:**

- Description: Button Shaper detects when the user presses a button. When a button is not pressed yet, the input signal $B_in=1$ and the current state, represented by the variable $State$ is in its initial form (INIT) marking that $B_out=0$ and the value on FPGA should not be registered yet. When the user presses the button and rst (reset) = 1, $B_in=0$ activating the input. It will set $B_out=1$ at the next moment that there is a rising edge on the clock (clk), this defines the value $Statenext=PULSE$, which will turn the next $State=PULSE$ if $rst=1$. When the pulse is over, the button will still be pressed but $B_out=0$ until the button is released so the module does not sent many high signals to the rest of the system at once. This defines the value $Statenext=WAIT$, which will turn the next $State=WAIT$ if $rst=1$. When the button is released, $B_in=1$ and this defines the value $Statenext=INIT$, which will turn the next $State=INIT$ if $rst=1$ for another opportunity to press the button and repeat the process. If $rst=0$, that means the reset is on as a safety procedure and this will automatically set $State=INIT$.
- Signal Definition:
 - B_in (signal_len: 1 bit) defines the detection of when the button is pressed whose signal is input in the module on the Combination Logic. When it is pressed, $B_in=0$. Otherwise, $B_in=1$.
 - clk (signal_len: 1 bit) is the clock of the module that is used for the sequential logic that establishes the width of a pulse to send one signal at a determined moment when the button is pressed.
 - $State$ (signal_len: 2 bits) is the 2-bit binary signal that shows the current state of the module and that may be changed (by using $Statenext$) when $rst=1$. The three possible states for the $State$ signal is $2'b00$ labeled as INIT, meaning the state before the button is pressed, $2'b01$ labeled as PULSE, which marks the only pulse in clk in which $B_out = 1$ and $2'b10$ labeled as WAIT, the phase that waits for the button to be released by the user.
 - $Statenext$ (signal_len: 2 bits) is the combinational output of the next-state logic that determines what the next value of $State$ will be at the next rising edge of clk . On each rising edge of clk , if $rst=1$, the value of $Statenext$ is loaded into $State$. Otherwise, $State$ is forced to its initial state, INIT ($2'b00$).
 - rst (signal_len: 1 bit) is the reset signal that will ensure $State=2'b00$ if $rst=0$.
 - B_out (signal_len: 1 bit) is a combinational output that, when $State$ equals PULSE ($2'b01$), outputs a single high pulse whenever, $B_in=0$ (button pressed) and clk is at a rising edge, only pulsing again after button is

released so it doesn't send a signal to the register many times while the user still presses the button.



- **Access Controller:**

- Description: The access controller restricts when the player can interact with the game. It waits for the users to enter the 4-digit password digit by digit, by inputting the 4-bit binary form of each digit through Password_digit and registering it after receiving a single high pulse from Third_button while the reset button is not pressed (rst=1). While the users are logged out, the controller outputs Logged_in=0 and Logged_out=1, keeping the system in the login phase and blocking game actions. It uses a 4-bit signal called State to determine which part of the login or gameplay process is active.
- The controller begins in state DIGIT1, where it waits for the first password digit. Each time Third_button sends a high pulse, it checks whether the current digit matches the expected value of the password and advances to the next state (DIGIT2, DIGIT3, DIGIT4). If any digit is incorrect, the signal Flag is turned low and remains low until the password is verified. In state VERIFY, the access controller checks Flag: if Flag=1, the password is correct and the controller goes to state PASSED; otherwise, it returns to DIGIT1 so the user can try again.
- In state PASSED, the user is officially logged in. The controller outputs Logged_in=1, Logged_out=0, keeps RNG_Gen_out=1 and Load_P_out=0, and sends a one-clock high pulse through time_reconfig so that the timer can be set to

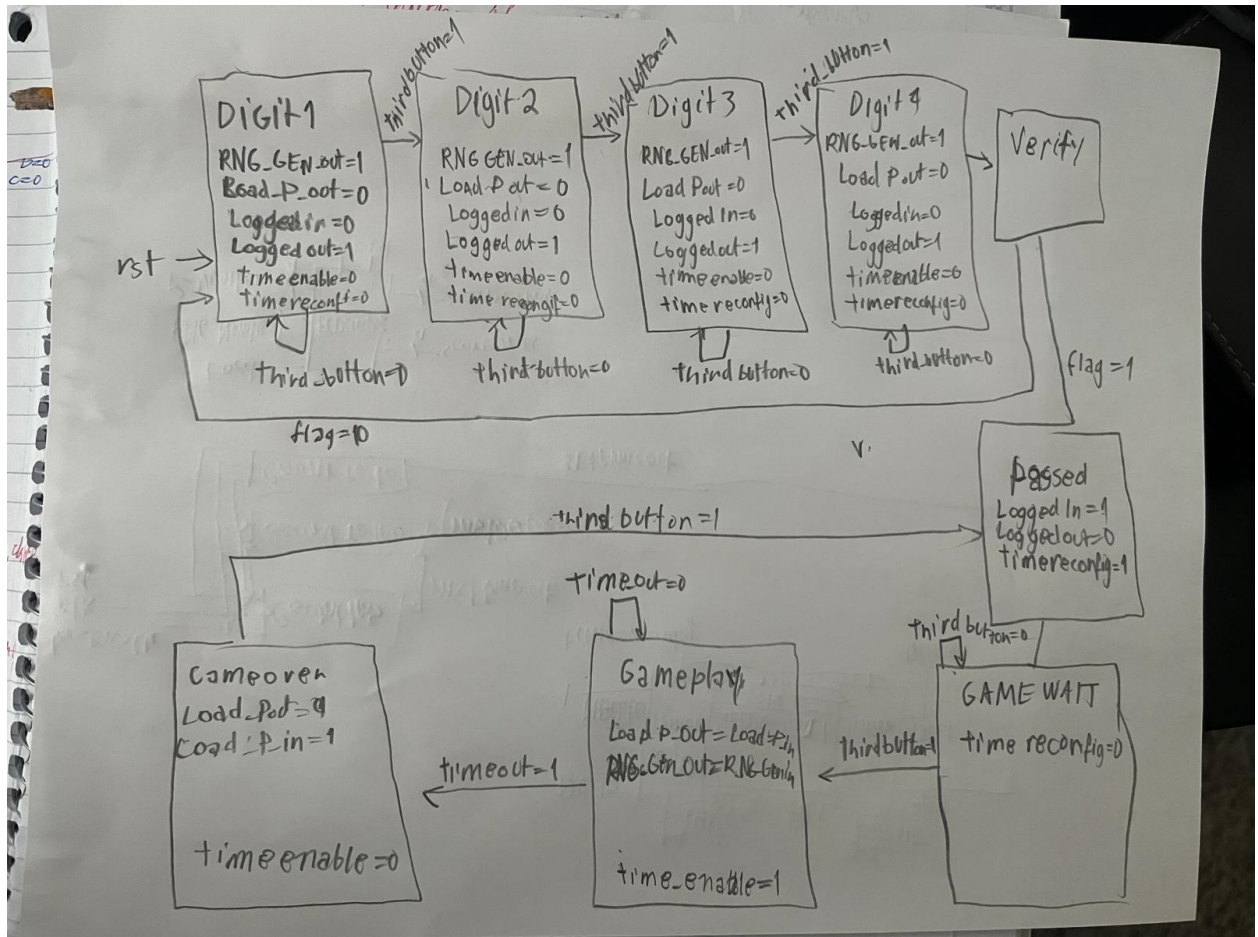
99. After that, it moves to state WAITGAME, where it waits for the users to press Third_button again to start the game.

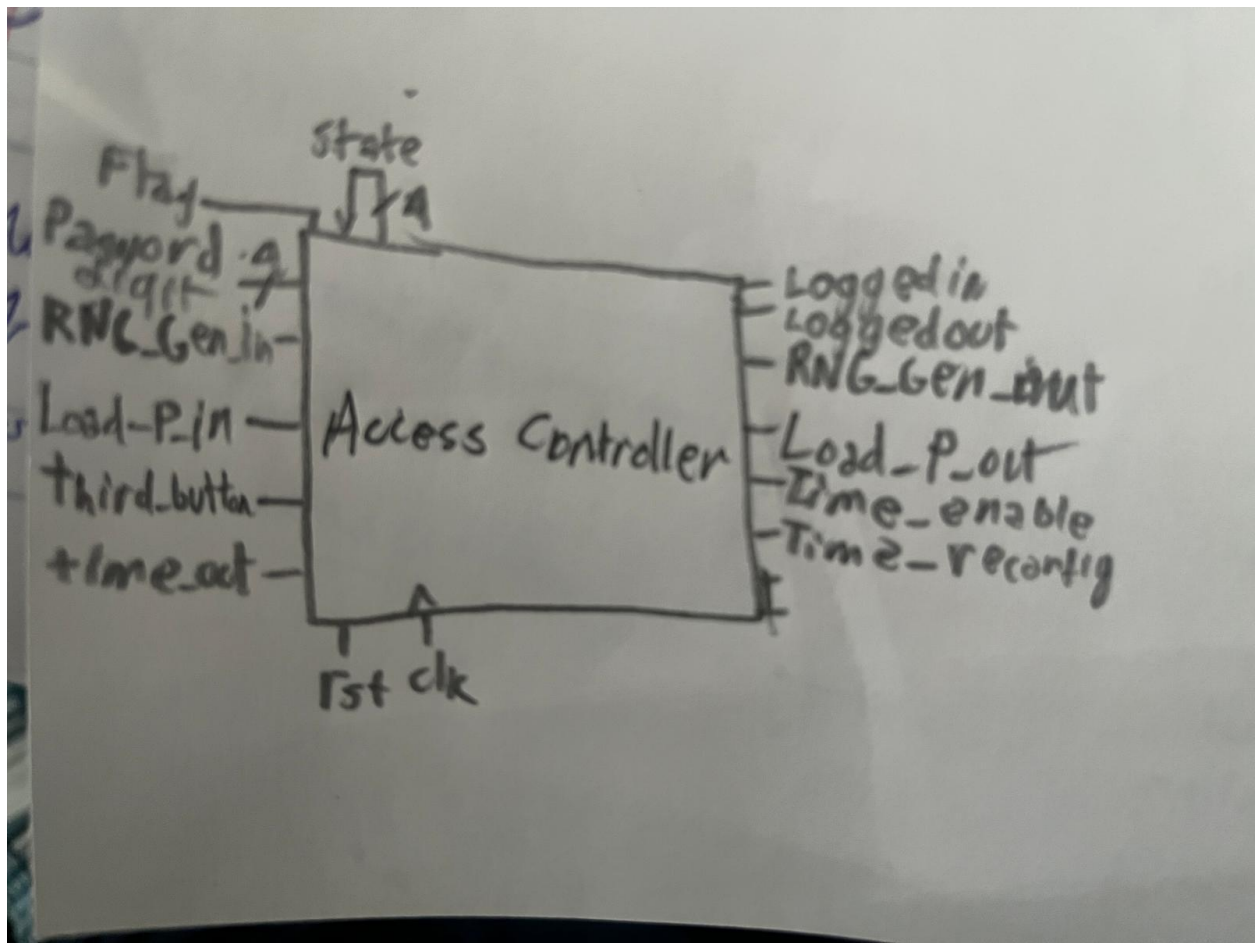
- In state WAITGAME, the user is logged in but the game is not running yet. The timer is not enabled (time_enable=0), RNG_Gen_out remains 1, and Load_P_out remains 0. When Third_button sends a high pulse, the access controller moves to GAMEPLAY.
- In state GAMEPLAY, the game is active. The controller outputs Logged_in=1, Logged_out=0, time_enable=1, and passes the incoming gameplay signals directly through the controller, so RNG_Gen_out = RNG_Gen_in and Load_P_out = Load_P_in. This allows the random number generation button and the player load button to operate during the game. The controller remains in this state until time_out=1, which indicates the game is over.
- In state GAMEOVER, the controller blocks the gameplay buttons again by setting RNG_Gen_out=1 and Load_P_out=0, while keeping the users logged in (Logged_in=1, Logged_out=0). The timer is disabled (time_enable=0). If the users press Third_button in this state, the controller goes back to PASSED, which sends another pulse through time_reconfig to reset the timer to 99 and prepares the system for a new game.
- Signal Definition:
 - Flag (signal_len: 1bit) defines a flag that starts with a high value when State=3'b000 (first digit still not entered). It turns low whenever Third_button=1 and Password_digit is different from the expected digit for that particular position in the password. In other words, the entered digit is wrong and the players cannot be logged in. It can be turned low when State=3'b000 (verifying first digit) and Password_digit is not equal the first digit expected, when State=3'b001 (verifying second digit) and Password_digit is not equal the second digit expected, when State=3'b010 (verifying third digit) and Password_digit is not equal the third digit expected and when State=3'b011 (verifying fourth digit) and Password_digit is not equal the fourth digit expected. If all digits entered are correct, flag will remain high. When State=3'b100 (determining if password is correct), if Flag is high, that means the password is correct and the players can log in. Logged_in will be turned high and Logged_out will be turned low, RNG_Gen_out will equal RNG_Gen_in and Load_P_out will equal RNG_Gen_in, and State will become 3'b101 (users are logged in). Otherwise, Logged_in will stay low and Logged_out will stay high, RNG_Gen_out and Load_P_out will stay low, and State will become 3'b000 (verifying first digit) for users to try a new password.
 - State (signal_len: 4 bits) is the 4-bit binary signal that shows the current state of the module and that may be changed when rst=1. The five possible

states for the State signal is 3'b000 labeled as DIGIT1, meaning the Access controller is verifying the first digit input by Password_digit when Third_button=1. State=3'b001 is labeled as DIGIT2, meaning the Access controller is verifying the second digit input by Password_digit when Third_button=1. State=3'b010 is labeled as DIGIT3, meaning the Access controller is verifying the third digit input by Password_digit when Third_button=1. State=3'b011 is labeled as DIGIT4, meaning the Access controller is verifying the fourth digit input by Password_digit when Third_button=1. State=3'b100 is labeled as VERIFY, meaning the Access controller is verifying if the password entered is correct by checking if Flag is low (incorrect) or high (correct). If Flag=0, State becomes 3'b000, otherwise it becomes 3'b101. Finally State=3'b101 is labeled as LOGGEDIN, meaning the Access controller logged the users in turning Logged_in high and Logged_out low and allowing the pulse signals from RNG_Gen_in and Load_P_in to be transmitted to RNG_Gen_out and Load_P_out respectively. If either rst=1 or Third_button=1 in this state, the users are logged out and State becomes 3'b000.

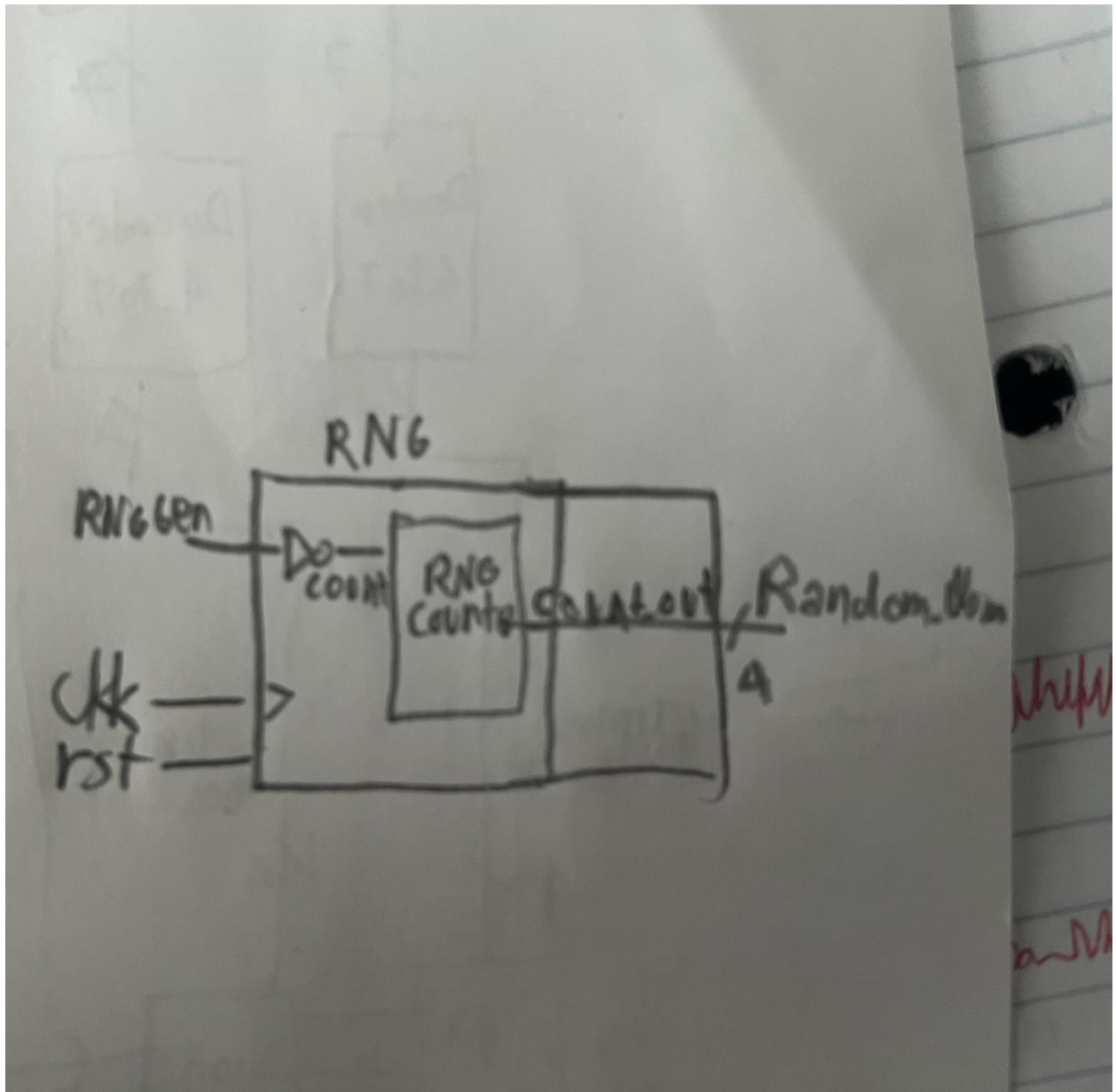
- Password_digit (signal_len: 4 bits) defines the 4-bit binary number input for password verification. Each 4-bit binary number entered is one digit from the password.
- Third_button (signal_len: 1 bit) is the signal that determines if a digit entered by Password_digit can be registered by the board and proceed the system for the input of the next digit when the reset button is not pressed (rst=1). It proceeds when a 1-pulse high signal comes from Third_button and when reset is not pressed. When the fourth digit is set by Password_digit and Third_button emits another 1-pulse high signal while rst=1, the access controller verifies if the 4-digit password is correct. Therefore, it is what changes the value of State variable while it still verifies the digits. If Third_button=1 and rst=1, it passes to the next state for verification of the next digit, otherwise it loops that state until the password button is pushed so Third_button inputs another 1-pulse high signal.
- RNG_Gen_in (signal_len: 1 bit) is the pulse signal that comes from Player 1's load button's button shaper. If users are logged in, the pulse will be transmitted to RNG_Gen_out. Otherwise, it will be canceled out by the access controller.
- Load_P_in (signal_len: 1 bit) is the pulse signal that comes from Player 2's load button's button shaper. If users are logged in, the pulse will be transmitted to Load_P_out. Otherwise, it will be canceled out by the access controller.

- RNG_Gen_out (signal_len: 1 bit) is the output that may send a pulse signal to Player 1's load register allowing Player 1's entered number to be displayed if users are logged in. Otherwise, it will remain low.
- Load_P_out (signal_len: 1 bit) is the output that may send a pulse signal to Player 2's load register allowing Player 2's entered number to be displayed if users are logged in. Otherwise, it will remain low.
- Logged_in(signal_len: 1 bit) is the signal that is high if the users are logged in and low otherwise.
- Logged_out (signal_len: 1 bit) is the signal that is high if the users are logged out and low otherwise.
- Time_reconfig (signal_len: 1 bit) is the signal that sends a high pulse to the digit Timer modules to set they digits to 9 initiating timers.
- Time_enable (signal_len: 1 bit) is the signal that enables the timer to run when high and not run when low.
- Time_out (signal_len: 1 bit) is the signal that when inputs high, it makes access controller block buttons when the game is over.
- rst (signal_len: 1 bit) is the reset signal that will ensure State=3'b000 if rst=0.
- clk (signal_len: 1 bit) is the system clock.





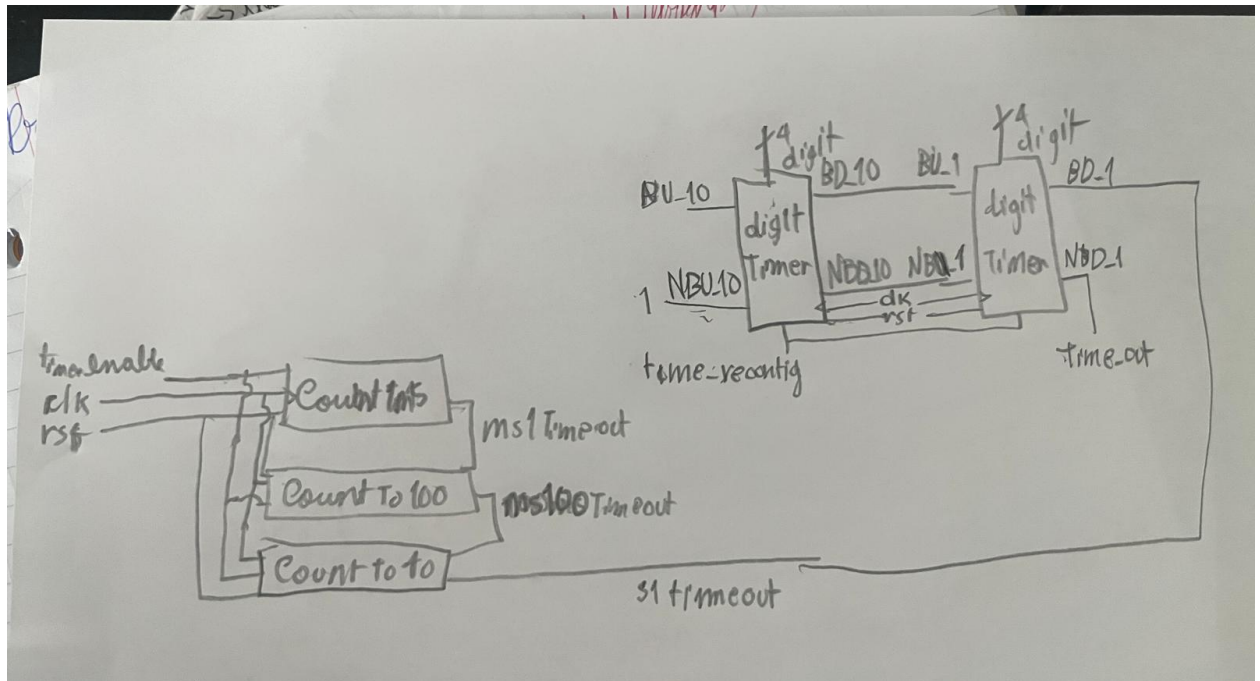
RNG:



- Description:
- The RNG (Random Number Generator) generates a random 4-bit number from 0 to 15 based on how long the player presses the Random Number Generation button (RNG Button) on the FPGA board. The 4-bit number will be decoded in its respective display and input to an adder for the sum. The RNG module receives the signal from Random Generation button through RNG_Gen as a low signal when pushed and high signal when released. Then the signal is inverted in count so count is high when RNG button is pressed and low otherwise and will be attached to the RNG_Counter to generate the random number based on how long the RNG Button is pushed (RNG_Gen=0 and count =1),. After the RNG button is released (RNG_Gen=1 and count =0), RNG directly outputs the current value of the random number in RandomNum, which is sent to the display and to the adder module for further processing.

- The RNG_Counter module is a 4-bit counter that generates the random number itself. When the reset button is pushed (rst = 0), the count_out remains 0. When reset is high (rst = 1) and count is high (count = 1), the count_out increments by 1 on each rising edge of the clock. When the count_out reaches its maximum value of 15, it becomes 0 again. When the count signal is low (count = 0), count_out stops incrementing and holds its current value generating the random number sent to Random_num.
- Signal Definition:
 - RNG_Gen (signal_len: 1 bit) This is the input signal from the Random Number Generation button. When the Random Generator button is pressed RNG_Gen = 0 enabling count_out to start incrementing and when button is released RNG_Gen = 1 interrupting the count_out from changing and outputting the value.
 - clk (signal_len: 1 bit) This is the system clock signal. The counter increments on the rising edge of clk when enabled.
 - rst (signal_len: 1 bit) This is the reset signal for the RNG system. When rst = 0, the counter is reset to 0 and remains inactive. When rst = 1, the counter operates normally depending on the count enable signal.
 - count (signal_len: 1 bit) This is the input signal to the RNG_Counter module (connected to count_s). When count = 1, the count_out increments at each clock cycle. When count = 0, count_out holds its value.
 - count_out (signal_len: 4 bits) This is the output of the RNG_Counter module. It represents the value to become the random number, from 0 to 15. This value continuously changes while the button is pressed and remains constant when the button is released.
 - RandomNum (signal_len: 4 bits) This is the final output of the RNG module. It is directly assigned from count_out and represents the generated random number. This value is displayed on the FPGA board and used as one of the inputs for the sum computation in the game.

Two_Digit Timer:



- Description: The two digit timer is a set of modules that controls the game's timer. It is composed by a one-second timer and two digit timers, one of them will send the tens portion of the amount of time player still has to a decoder to output in a display another will send the ones portion of the amount of time player still has to a decoder to output in a display.
- The one-second timer allows the time to count each second of time limit for one round of game on the FPGA board to decrease the timer by 1 every second during the gameplay. It outputs one high pulse through s1Timeout after one second (50,000,000 cycles from the clock system) when the time_enable input received from the access controller is high (time_enable = 1) and the reset button is not pressed (rst = 1). Count1ms module takes 50000 clock cycles before outputting one high pulse to CountTo100 module. CountTo100 module inputs 100 high pulses from Count1ms before outputting one high pulse to CountTo10 module. CountTo10 module inputs 10 high pulses from CountTo100 before outputting one high pulse to outside the one-second timer. By multiplication, this fills the total of 50,000,000 clock cycles or one second before outputting a pulse signal to the two digit timers to be decreased by one, consequentially decreasing the timer shown on the displays as well. When either reset is pressed (rst = 0) or enable is turned low (time_enable = 0), no pulse signal is transmitted between the modules and to outside the timer, which means it cannot make the game's timer run, making it remain 0.
- The ones digit Timer will receive the high pulse from time_reconfig from access controller to be initiate digit to 9. NBD_1 signal (No_Borrow_down) will be low since its value is not zero meaning there is still time left. During

the game, when it receives a high pulse from s1Timeout attached to its BD_1 signal (Borrow_Down), it will decrease the digit by 1. This repeats until it is 0. When that happens it will send a high pulse through the BU_1 signal to the tens digit Timer to set its digit back to 9 and restart the decrement, simulating taking one unit from tens to the ones. If NBU_1 signal (No_Borrow_up) is low, it means the tens digit still has units available to give to the ones digit and ones is set back to nine and tens is decreased by 1. Otherwise, it means tens cannot give more unities and therefore the ones digit remains 0, making NBD_1 signal (No_Borrow_down) becomes high and by being attached to the time_out wire it will send a high pulse to the access controller to block the buttons indicating end of game and will make the total score be output in the central displays.

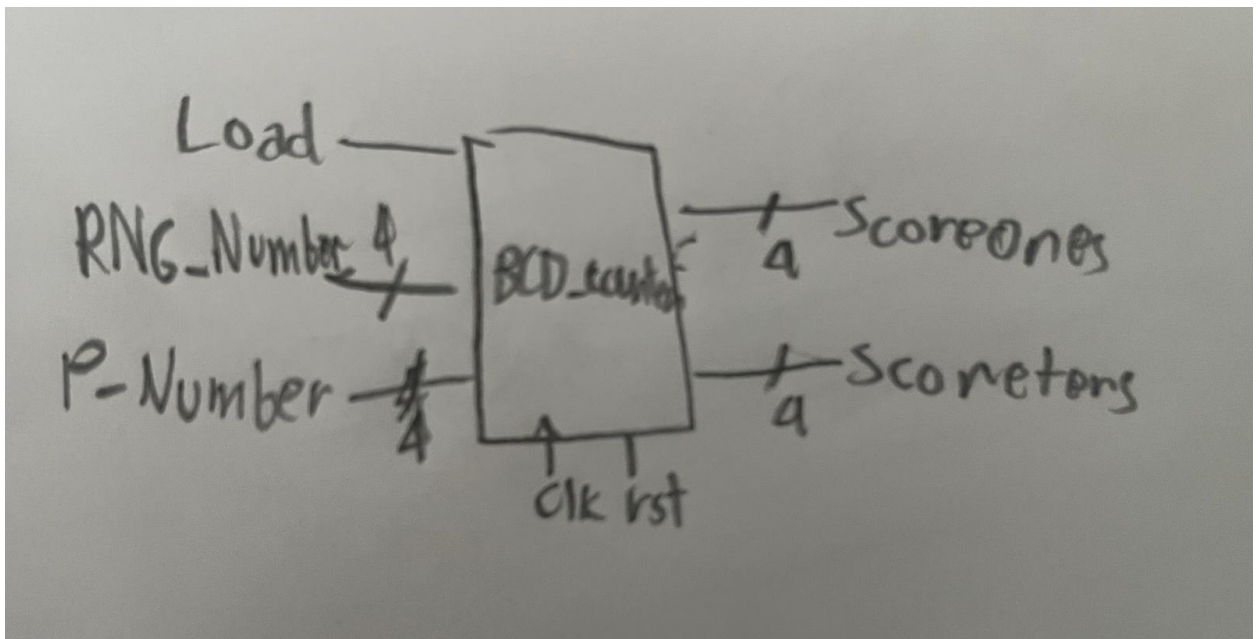
- The tens digit Timer will receive the high pulse from time_reconfig from access controller to be initiate digit to 9. NBD_10 signal (No_Borrow_down) will be low since its value is not zero meaning there is still units left in it. During the game, when it receives a high pulse from BD_10 signal (Borrow_Down) attaches to the ones digit's BU_1, it will decrease the digit by 1. This repeats until it is 0. When that happens, the tens cannot give more unities (NBU_10 is always high) and therefore the ones digit remains 0, making NBD_10 (No_Borrow_down) becomes high and by being attached to the ones digit's NBU_1 wire it will send a high pulse to it to prevent a high pulse from coming from BD_10 to try to borrow from it. When the tens digit cannot give more units for the ones digit that is already 0, it means the time is up.
- Signal Definition:
 - Time_reconfig (signal_len: 1 bit) sends a high pulse to set the digit in each digit timer to 9 before game starts
 - Time_out(signal_len: 1 bit) it determines if access controller should block the buttons if game is over. If both timers indicate digit equal to 0, it will send a high signal to block buttons and low not to block.
 - Time_enable (signal_len: 1 bit) is the input signal of the modules Count1ms, CountTo100 and CountTo10. When enable input a high signal to each module, after 50,000,000 clk cycles, Count1ms transmits one high pulse through ms1Timeout to CountTo100. After receiving 100 high pulses from ms1Timeout, CountTo100 transmits one high pulse through ms100Timeout to CountTo10. After receiving 10 high pulses from ms100Timeout, CountTo10 will output one high pulse through s1Timeout to the next module. In total, s1Timeout sends 1 high pulse after 50,000,000 time cycles of high enable input, as it

follows the multiplication of 10 ms100Timeout high pulses, 100 ms1Timeout high pulses and 50,000,000 clk cycles. When enable is turned off, no high pulse is transmitted through ms1Timeout, ms100Timeout and s1Timeout

- clk (signal_len: 1 bit) is the clock of the module that is used for the sequential logic that establishes the width of the pulses that are output by ms1Timeout, ms100Timeout and s1Timeout.
- rst (signal_len: 1 bit) is the reset signal that will reset the system and keep the counter off. When either rst = 0 or enable = 0, ms1Timeout, ms100Timeout and s1Timeout will remain low without outputting any pulse and the modules' internal counters will stay at 0 and will not make the timer run. Otherwise, if both rst = 1 and enable turns high, s1Timeout will output one high pulse after 50,000,000 clk cycles.
- ms1Timeout (signal_len: 16 bits) is the output of Count1ms module that outputs a high signal after 50,000,000 clk cycles when rst = 1 and enable = 1 and remains low otherwise.
- ms100Timeout (signal_len: 7 bits) is the output of CountTo100 module that outputs a high signal after it inputs 100 high pulses from ms1Timeout when rst = 1 and enable = 1 and remains low otherwise.
- s1Timeout (signal_len: 4 bits) is the output of CountTo10 module that outputs a high signal after it inputs 10 high pulses from ms100Timeout when rst = 1 and enable = 1 and remains low otherwise.
- digit (signal_len: 4 bits) it is the digit currently displayed by the digitTimer
- tens_digit (signal_len: 4 bits) it is the digit currently displayed by the digitTimer corresponding to the tens portion of the timer
- ones_digit (signal_len: 4 bits) it is the digit currently displayed by the digitTimer corresponding to the ones portion of the timer
- BD_1 (signal_len: 1 bit) it is attached to the s1Timeout signal and it will make digit from ones timer, be decreased by 1.
- BU_1 (signal_len: 1 bit) if NBU_1 is low and the digit inside the ones timer is 0, it will send a high pulse for the tens digit to make the tens digit be decreased by 1 and the ones digit be set back to 9.

- NBU_1 (signal_len: 1 bit) determines if the ones digit can borrow units from the tens digit, low if yes, high if not. If NBU_1 is high, BU_1 will remain low even if digit is 0 and will make NBD_1 high for time_out to send a high signal to the access controller to make the game stop.
- NBD_1 (signal_len: 1 bit) it is attached to the time_out signal and it will be low if time is still not up and high otherwise.
- BD_10 (signal_len: 1 bit) it is attached to the BU_1 signal and it will make digit from tens timer be decreased by 1 if it is not 0 yet.
- BU_10 (signal_len: 1 bit) it will remain low since it will not make the tens digit borrow from another unit since there is no hundredths digit
- NBU_10 (signal_len: 1 bit) will remain high so BU_10 stays low indicating there is no possibility to borrow from a hundredths digit.
- NBD_10 (signal_len: 1 bit) it is attached to the NBU_1 signal and it will be low if there is still units in the tens digit making it possible to lose for the ones digit and high otherwise, not allowing the ones digit to send a high pulse from BD_10 ten requesting another unit.

BDC Counter:



- Description: BCD Counter sums the number of times the sum matches the target of 15 during the game. It inputs the random number generated by the RNG called

RNG_Number and sums it to the 4-bit binary number obtained by the combination of switches from Set 2 (Figure 1) called P_Number and then it waits for a high pulse from the Load Button (Figure 1) when the game is on. After the high pulse, if the sum between RNG_Number and P_Number is 15, then it increases 1 to Score_ones output, which indicates the unit portion of the total score. If Score_ones is 9 and Score_tens is less than 9, if another point is to be added, Score_tens is increased by 1 and Score_ones is set back to 0. If both Score_ones and Score_tens are 9, there is no change because this is the maximum score possible.

- Signal Definition:
 - RNG_Number (signal_len: 4 bits) is the 4-bit binary number input by the RNG.
 - P_Number (signal_len: 4 bits) is the 4-bit binary number input by the four switches in player's set on FPGA.
 - clk (signal_len: 1 bit) is the system clock.
 - rst (signal_len: 1 bit) is the reset signal that is high whenever the reset button is pressed.
 - Time_reconfig (signal_len: 1 bit) is the signal that reconfigures the timer to 99 and what reset the total score back to 0.
 - Load (signal_len: 1 bit) is the signal that will allow for a point to be added to the total score (defined in two digits by Score_ones and Score_tens) if the sum between RNG_Number and P_number is 15.
 - Score_Ones (signal_len: 4 bits) is the ones portion of the total score and while it is less than 9 it will be increased by 1 when the sum between RNG_Number and P_number is 15. When it equals 9 and the sum between RNG_Number and P_number is 15, it becomes 0 when Score_tens is less than 9 and makes Score_tens be increased by 1 and stays 9 by both Score_tens and Score_ones are 9.
 - Score_Tens (signal_len: 4 bits) is the tens portion of the total score and while it is less than 9 it will be increased by 1 when the sum between RNG_Number and P_number is 15 and Score_tens current value is 9. If it is currently 9 and the sum between RNG_Number and P_number is 15, nothing changes.

Multiplexer:

- RESULTS:

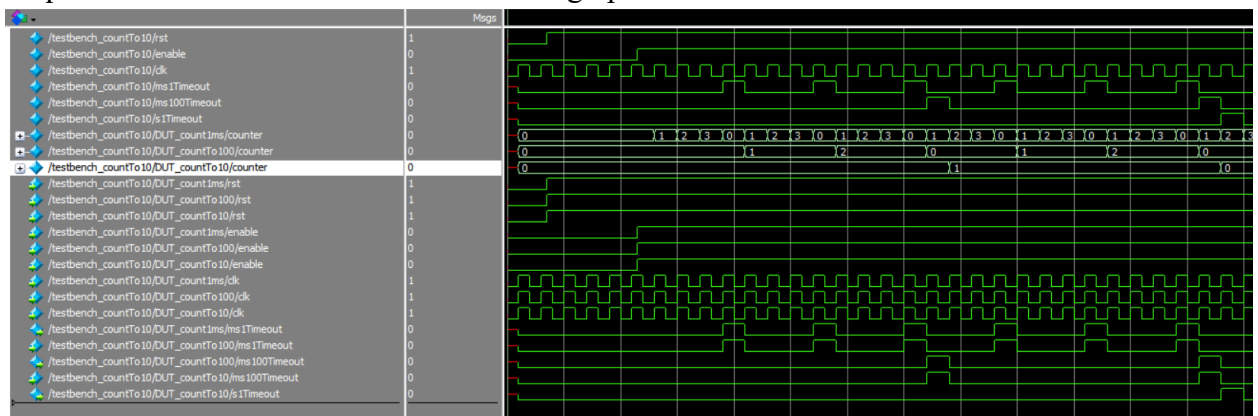
Signal	Value	Waveform
Mag	0	
/testbench_RNG/RNG_Gen	0	
/testbench_RNG/jk	1	
/testbench_RNG/rst	1	
/testbench_RNG/Random_run	1	
/testbench_RNG/OUT_RNG/RNG_Gen	0	
/testbench_RNG/OUT_RNG/jk	0	
/testbench_RNG/OUT_RNG/rst	1	
/testbench_RNG/OUT_RNG/Random_run	0001	
/testbench_RNG/OUT_RNG/count_5	1	
/testbench_RNG/OUT_RNG/count_out_5	0	
/testbench_RNG/OUT_RNG/Counter/jk	0001	
/testbench_RNG/OUT_RNG/Counter/rst	1	
/testbench_RNG/OUT_RNG/Counter/count	1	
/testbench_RNG/OUT_RNG/Counter/count_out	0001	

Figure 3. RNG simulation

The results shown in Figure 3 simulate a RNG after reset button is released (rst=1) and RNG button is pushed twice count and count_out start as 0. When RNG button is pushed, RNG_Gen becomes low and count becomes high allowing count_out to change at each rising clock edge of clk making Random_num change along. When RNG button is released for the first time (RNG_Gen becomes high and count becomes low), count_out and Random_num become 11 and when RNG button is pushed again, they grow until 15 then restart as 0 and output 4 when RNG button is released again.

- One-Second Timer

For simplification, the output of the One-Second Timer counts only 24 clock cycles instead of 50,000,000. The Count1ms outputs ms1Timeout after 4 clock cycles, the CountTo100 outputs ms100Timeout after 3 ms1Timeout high pulses and CountTo10 outputs s1Timeout after 2 ms100Timeout high pulses.



The result in Figure 1 shows the first cycle of the counter. The counters of each module remains equal to 0 and no pulse comes from ms1Timeout, ms100Timeout or s1Timeout while enable is low and rst is low. When both rst and enable are turned high, the counters begin to change their values. Count1ms's counter changes 4 times before CountTo100's counter changes, CountTo100's counter changes 3 times before CountTo10 changes, and CountTo10's counter changes 2 times before s1Timeout outputs a single pulse.

Ms1Timeout outputs a pulse after every 4 clock cycles. Ms100Timeout outputs a pulse after every 3 Ms1Timeout high pulses. S1Timeout outputs one high pulse after 2 Ms100Timeout high pulses.

Load inputs a high pulse, D_out is updated to D_in's most recent value, 4'b0111. When rst=0 (reset button is pushed), D_out becomes 4'b0000 indicating that the system was reset.

- Button Shaper

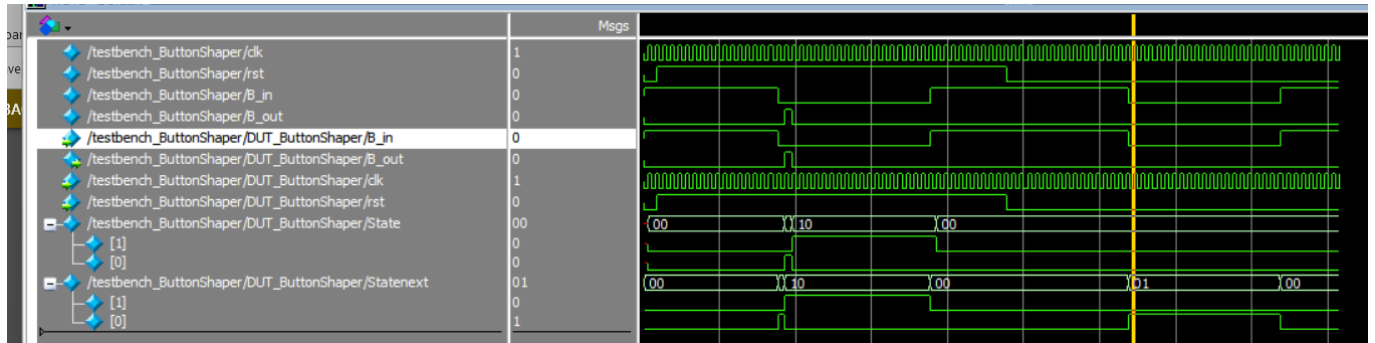


Figure 6. Simulation results.

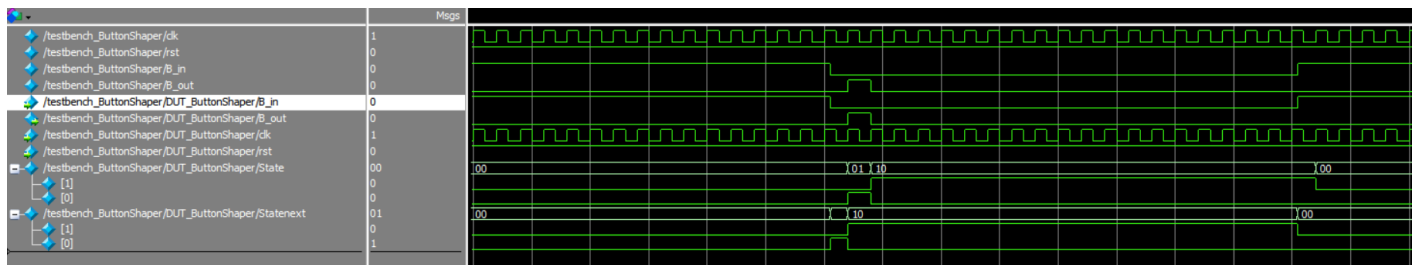


Figure 7. Results when rst=1

The results shown in Figure 7 simulate a button shaper when the button was initially not pressed ($B_in = 0$, $State = INIT$ or $2'b00$). Because $rst=1$ initially, that means that when the button is pressed, B_in becomes 1, and at the next time the clock is at a rising edge, B_out emits a single pulse ($B_out=1$ when $State = 2'b01$ or $PULSE$). When the pulse is over $B_out=0$ again ($B_out=1$ when $State = 2'b10$ or $WAIT$). And the State only goes back to $INIT$ or $2'b00$ at the first rising edge of the clock when B_out is back to 0.

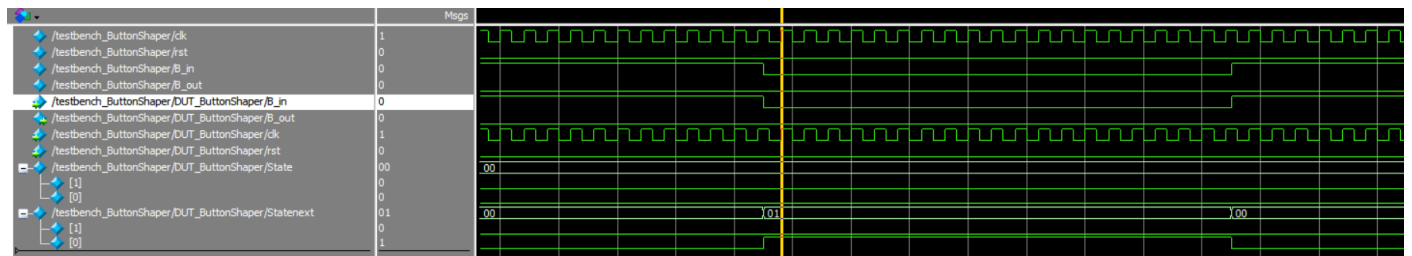


Figure 8. Results when rst=0.

Next, in Figure 8, rst is set to 0 and the simulation shows that when the button is pressed, B_out does not send a high pulse because State is always at the initial condition, State=2'b00 or INIT.

- Access Controller:

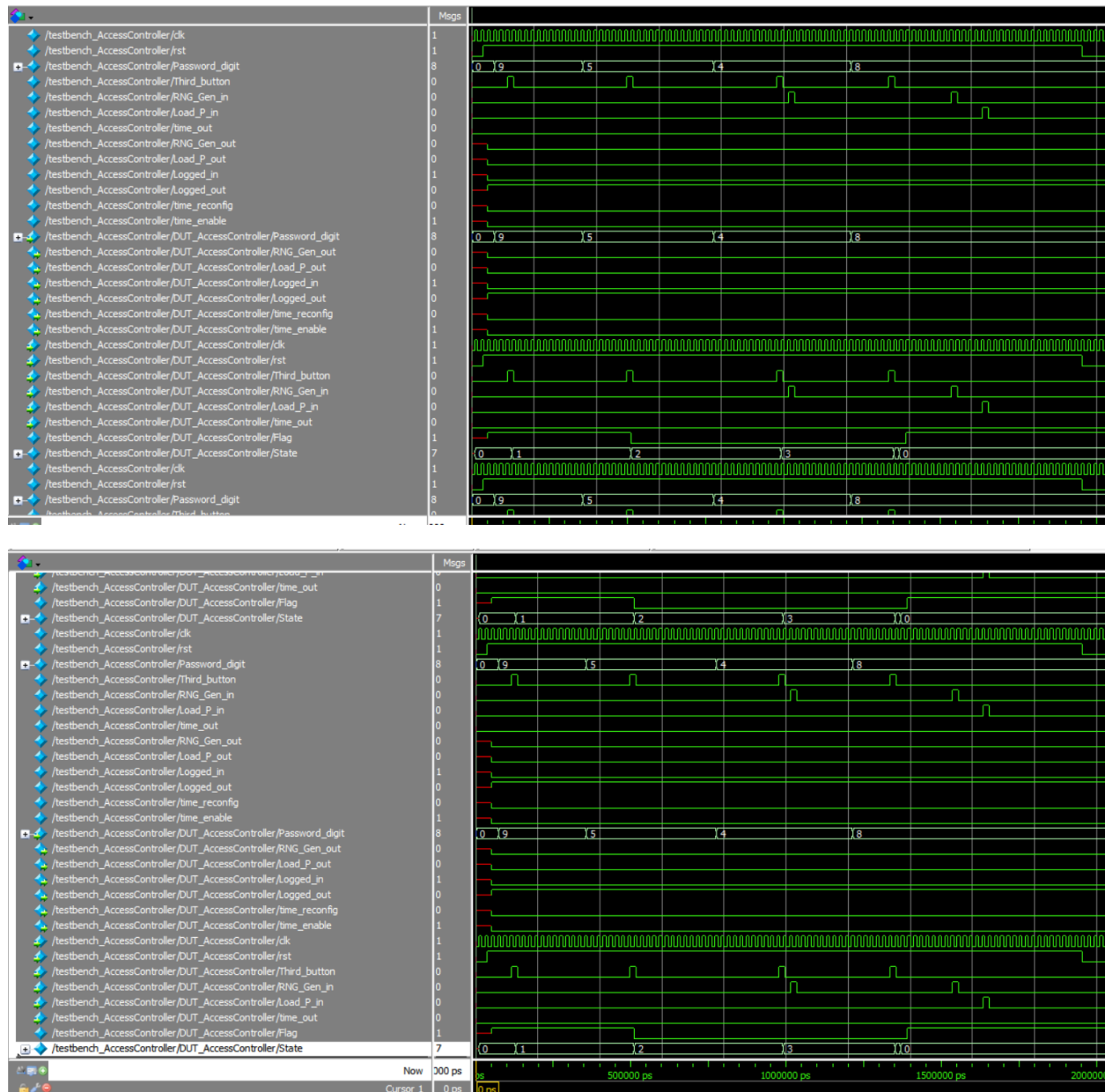


Figure 2. Wrong password case

The right password to log in is 9048. Figure 8 shows a simulation of the access controller when the user enters a wrong password. Initially the Logged_in signal is low and Logged_out signal is high, indicating that the users are logged out. Time_out, Time_enable and Time_reconfig are all low since those features will only be important when user is logged in. Additionally, Flag is high

State=0, meaning the controller is waiting for user to enter the first digit of the password. The user inputs 9 in Password_digit and sends a high pulse signal in the meantime by pushing the password button. Entering the digit 9 first. Then State becomes 1 meaning it is ready to receive the second digit. The user then enters 5 in Password_digit before sending another high pulse signal from Third_button and change State to 2. It registers number 5, but that is not the correct second digit. This makes Flag becomes low, and the controller will not allow those users to log in even if they finish the password. In State=2, the controller is waiting for the third digit. Password_digit inputs the digit 4 and Third_button sends the high pulse to register that digit and change State to 3. It then simulates Player 1 pushing their load button when the password is incomplete, which makes RNG_Gen_in input a pulse high signal to the controller, but since the State is not 5 (logged in) that signal does not affect RNG_Gen_out. It remains low. Password_digit inputs the digit 8 and Third_button sends the high pulse to register the last digit and change State to 4 in which the password is verified. Because Flag is low, it means that password is wrong. Instead of changing the State to 5, which would mean the players are logged in, it regresses back to state 0 because the authentication was unsuccessful. Logged_in stays low and Logged_out stays high and Flag is set back to high. It simulates the players trying to play the game despite still being logged out by making RNG_Gen_in and Load_P_in input more high pulse signals (simulating them pushing their load buttons), but RNG_Gen_out and Load_P_out are still low and will not transmit those pulse signals to a load register to display their numbers. Player's must try to authenticate again. The reset button is pushed to reset the system for the second case, making rst low.

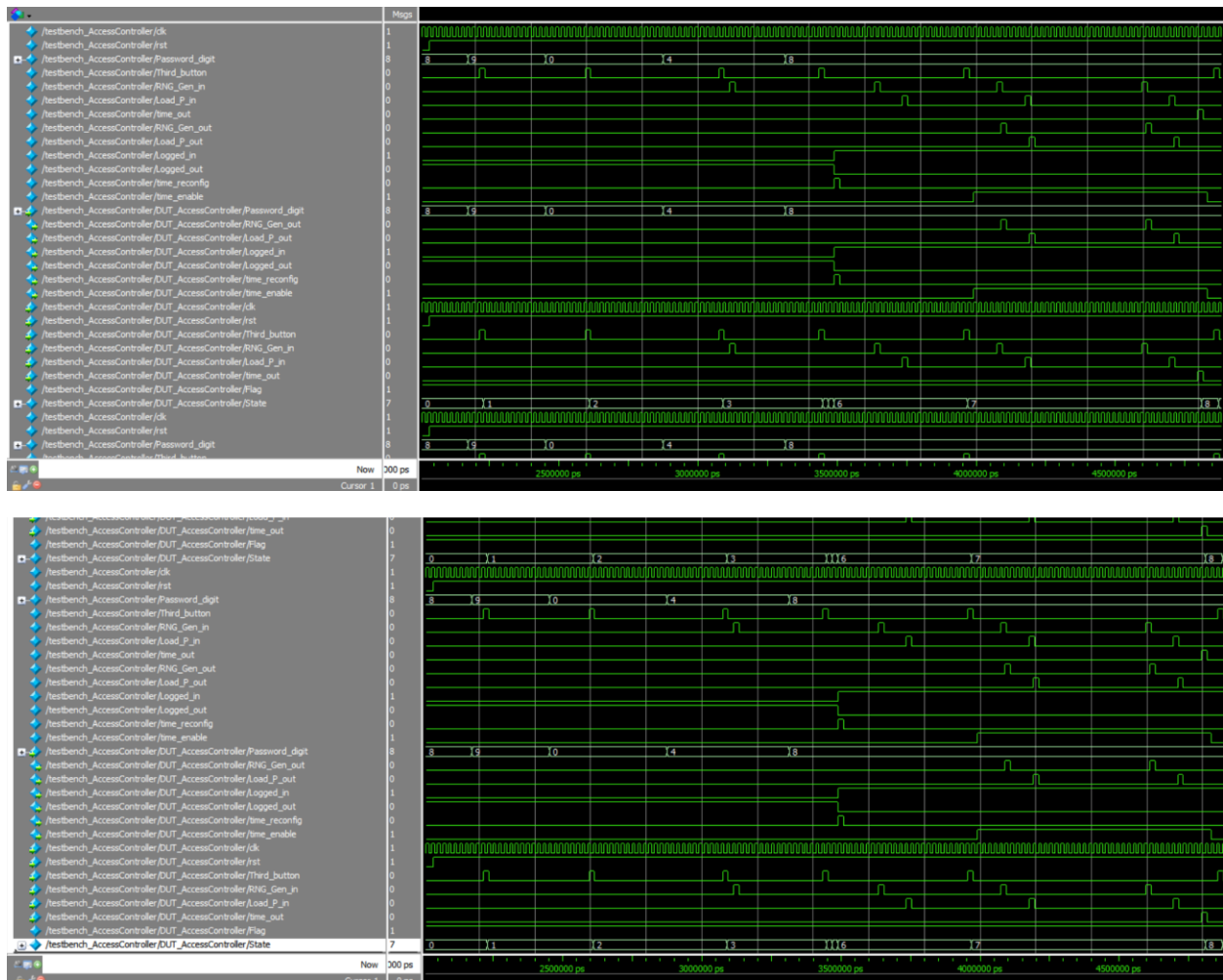


Figure 3. Right password case

Figure 9 shows a simulation of the access controller when the user enters the correct password. Initially the Logged_in signal is low and Logged_out signal is high, indicating that the users are logged out. Additionally, Flag is high, State=0, meaning the controller is waiting for user to enter the first digit of the password. The user inputs 9 in Password_digit and sends a high pulse signal in the meantime by pushing the password button. Entering the digit 9 first. Then State becomes 1 meaning it is ready to receive the second digit. The user then enters 0 in Password_digit before sending another high pulse signal from Third_button and change State to 2. It registers number 0. In State=2, the controller is waiting for the third digit. Password_digit inputs the digit 4 and Third_button sends the high pulse to register that digit and change State to 3 so the controller waits for users to enter the last digit. Password_digit inputs the digit 8 and Third_button sends the high pulse to register the last digit and change State to 4 in which the password is verified. Because Flag remained high all the time, it means that password it right. State becomes 5 indicating the users are now logged in. A high pulse outputs in time_reconfig to set timer to 99. Flag stays high, the Logged_out signal becomes low and Logged_in signal becomes high. It simulates the player trying to play the game by pushing their buttons and make RNG_Gen_in

and Load_P_in input more high pulse signals but only after another high pulse comes from Third button (start game) that time_enable becomes high RNG_Gen_out and Load_P_out transmit those pulse signals to be used by a load register to display their numbers.

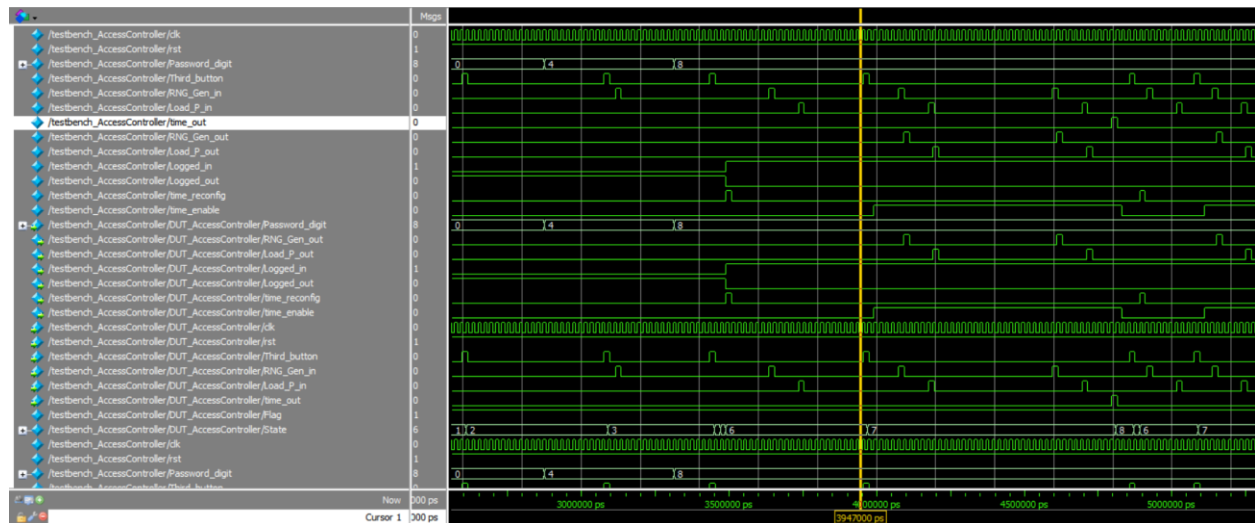


Figure 4. Game over and restart

Figure 10 shows the end of the simulation in which shows the access controller receiving a high pulse from Time_out that turns Time_enable low making the timer stop. When RNG_Gen_in and Load_P_in input high pulses RNG_Gen_out and Load_P_out remain low since game is over. When Third_button sends another high pulse signal (Restart game), another high pulse comes from Time_reconfig to set timer back to 99 to start a new game. RNG_Gen_out and Load_P_out remain low. When another high pulse comes from Third_button (Start game), Time_enable is set back high and RNG_Gen_out and Load_P_out output RNG_Gen_in and Load_P_in high pulses for a new game.

- Decoder

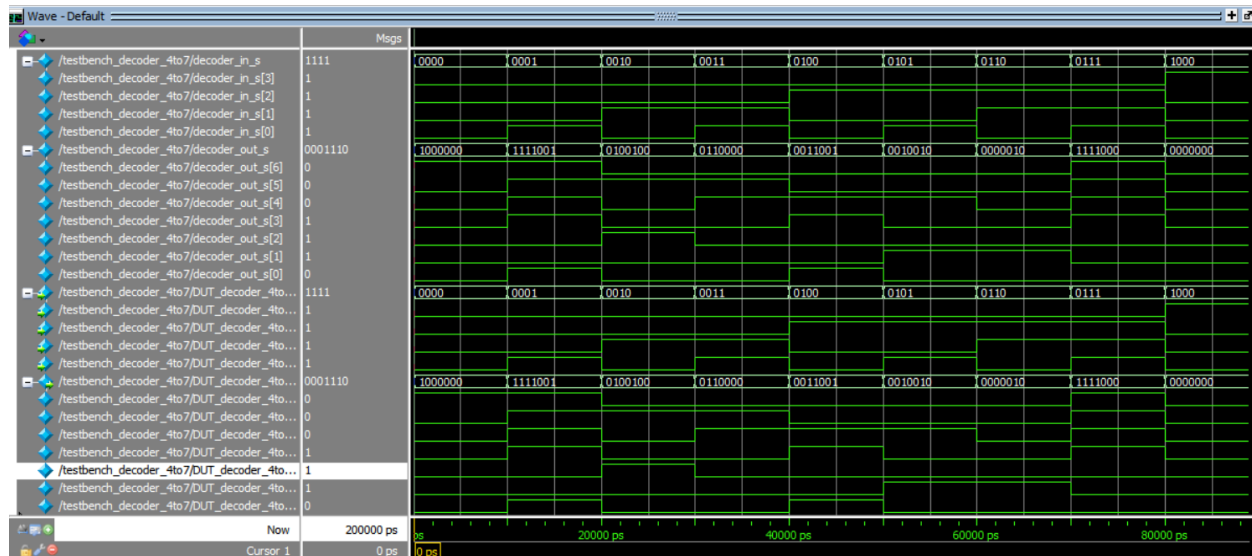
For the model of the decoder on Figure 3, it was performed the test to see if an input of a binary number entered would generate the correct corresponding combination of 7 bits on the output that will turn on the correct segments on the displayer to show the equivalent hexadecimal number. It should display as it is shown in Table 1

Table 1. Results of decoder_4to7 simulation. The first row indicates the theoretical hexadecimal numbers output on displayer; the second row shows the binary number input and the third row shows the 7-bit combination output to generate the hexadecimal value

0	0000	1000000
1	0001	1111001
2	0010	0100100

3	0011	0110000
4	0100	0011001
5	0101	0010010
6	0110	0000010
7	0111	1111000
8	1000	0000000
9	1001	0011000
A	1010	0001000
b	1011	0000011
C	1100	1000110
d	1101	0100001
E	1110	0000110
F	1111	0001110

Each time I change the input there is a new wave showing a new 7-bit output as expected. For the output bits equal to zero, the segment will be on and for bits equal to one, the segment will be off. The case where output is 111111_2 is the mandatory default case in the case statement inside the decoder model in which the display will be entirely off.



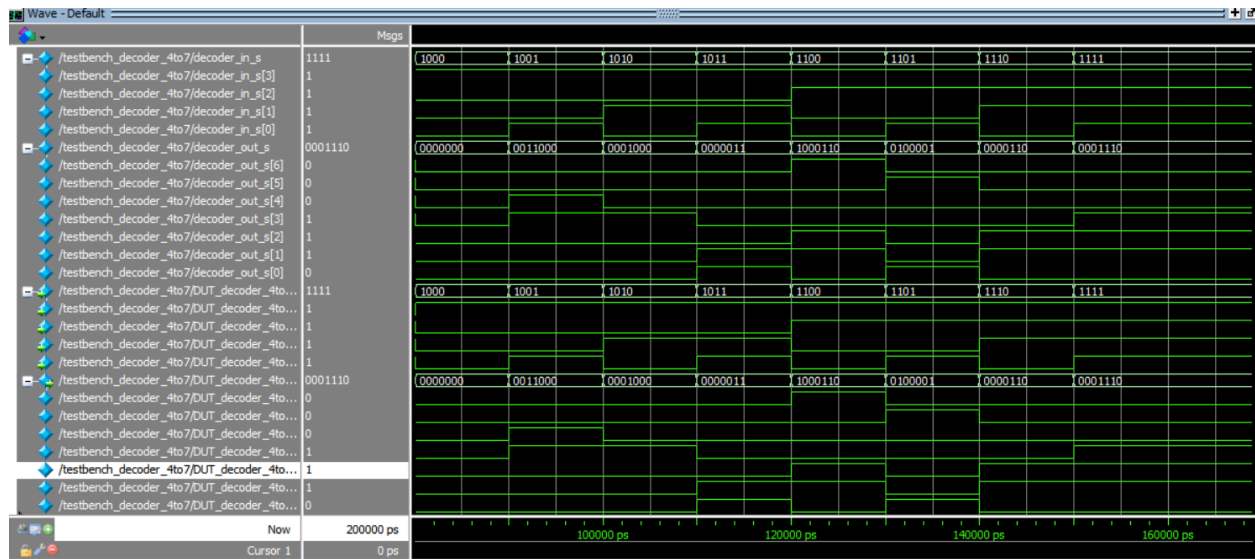
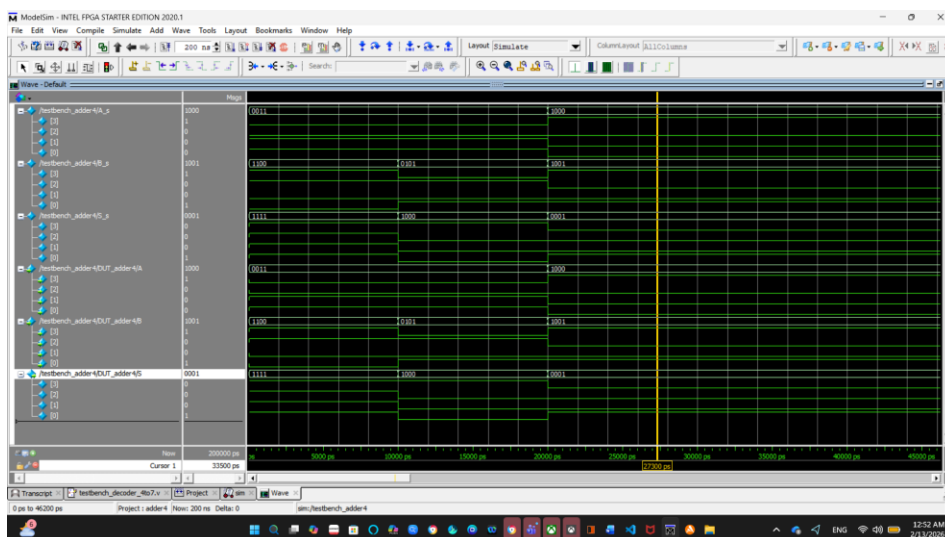


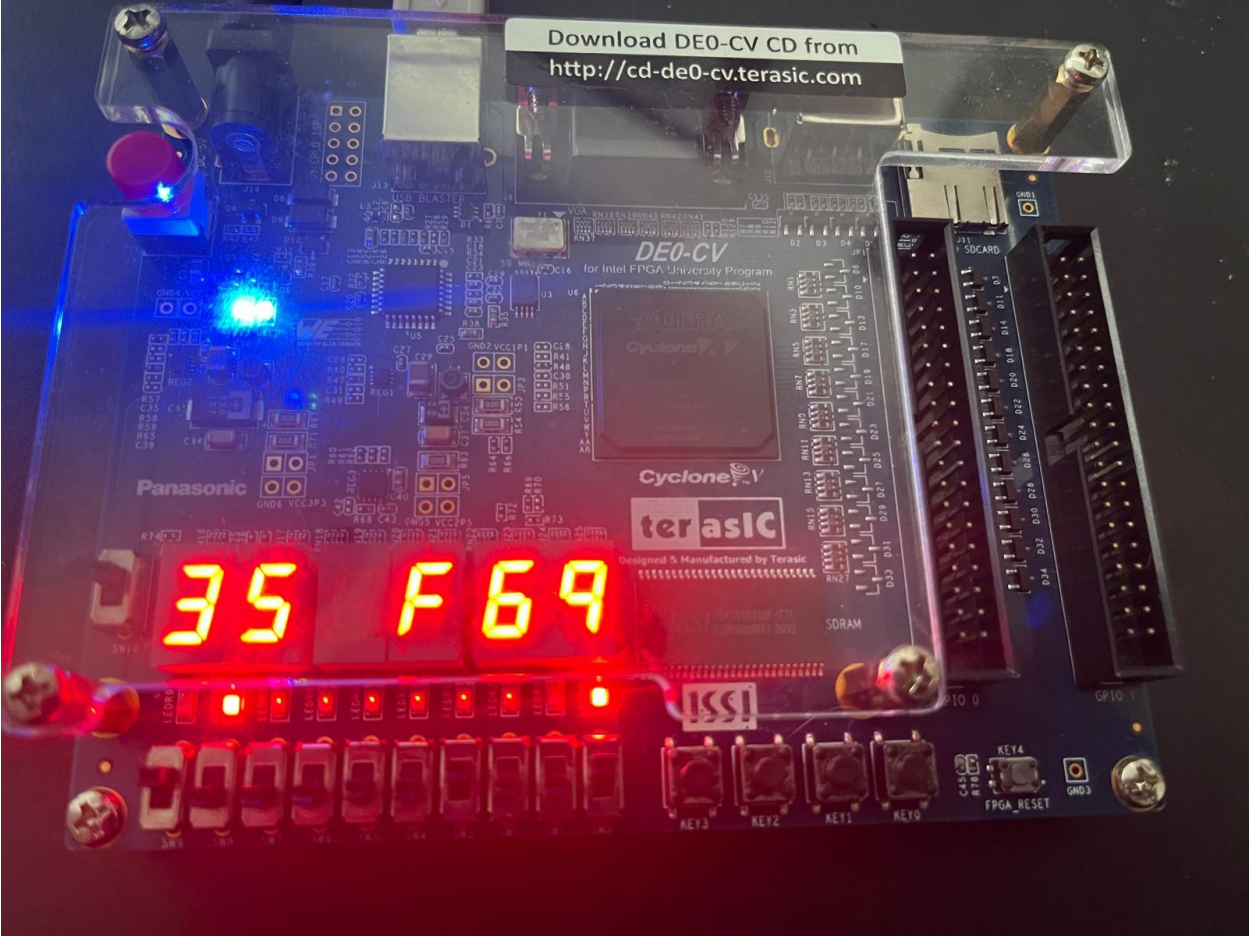
Figure 5. Simulation results for the decode_4to7 model.

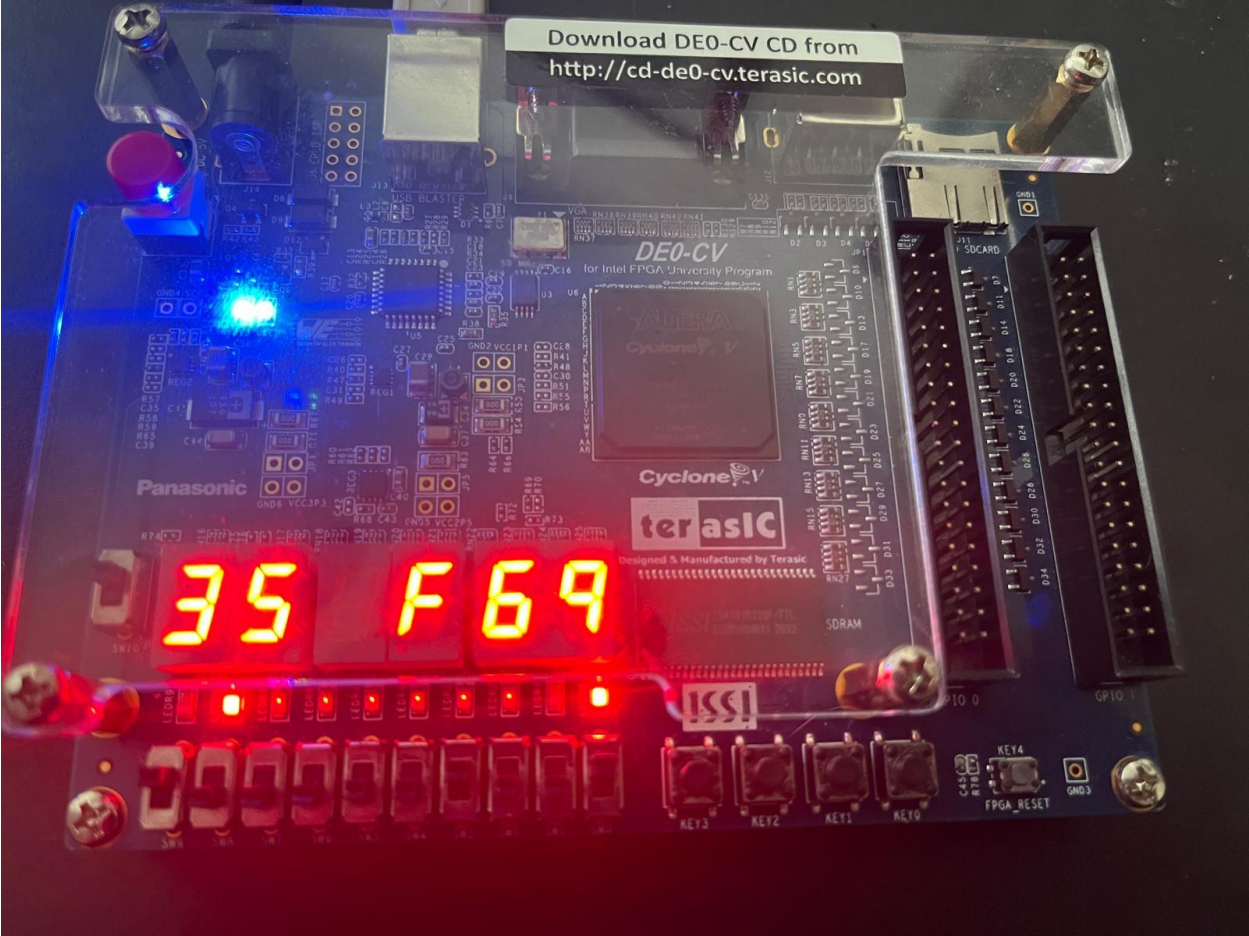
- Adder

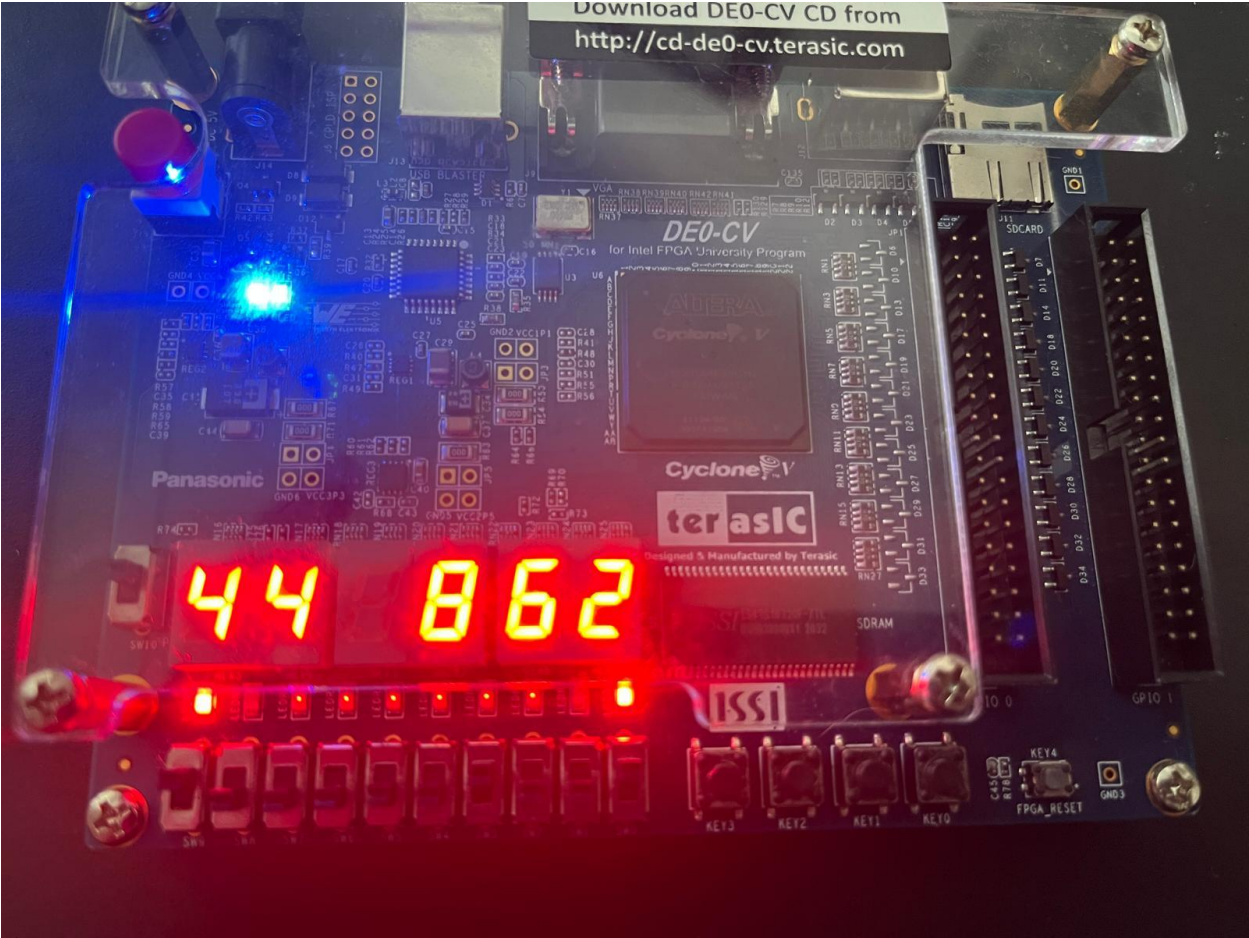
For the adder model on Figure 4 I tested three cases of 4-bit binary numbers entered by the players. For the first wave, it assumes Player 1 entered 0011_2 and Player 2 entered 1100_2 . The sum will result in 1111_2 , matching the targeted number of f_{16} . For the second wave Player 1 entered 0011_2 and Player 2 entered 0101_2 . The sum will result in 1000_2 , not matching the targeted number of f_{16} . For the third wave Player 1 entered 1000_2 and Player 2 entered 1001_2 . The sum will result in 0001_2 , not matching the targeted number of f_{16} .

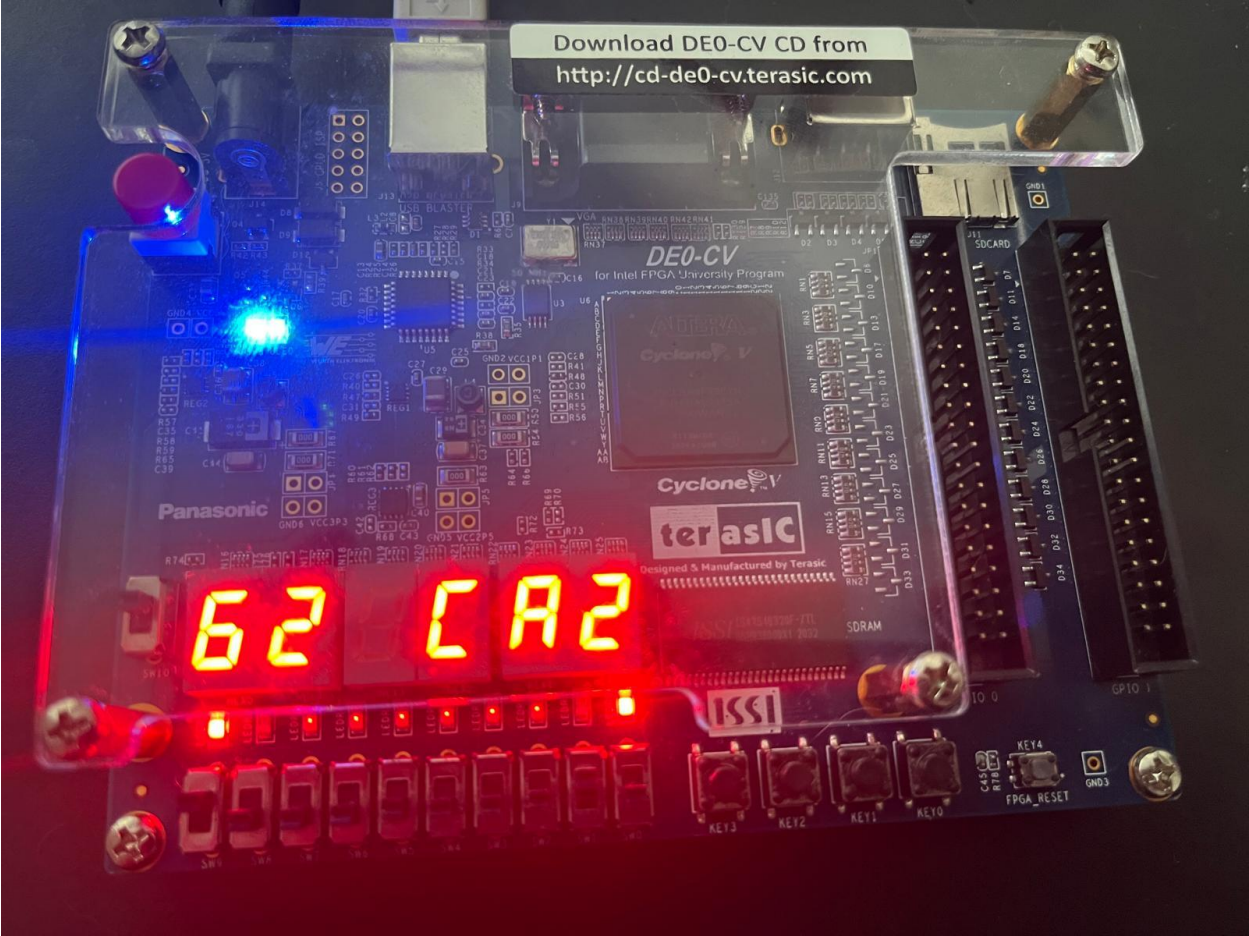


FPGA Board Testing Results









Download DE0-CV CD from
<http://cd-de0-cv.terasic.com>

DE0-CV
for Intel FPGA University Program

ALTERA
Cyclone V

terasic
Designed & Manufactured by Terasic

Panasonic

62 CA2

ISSI

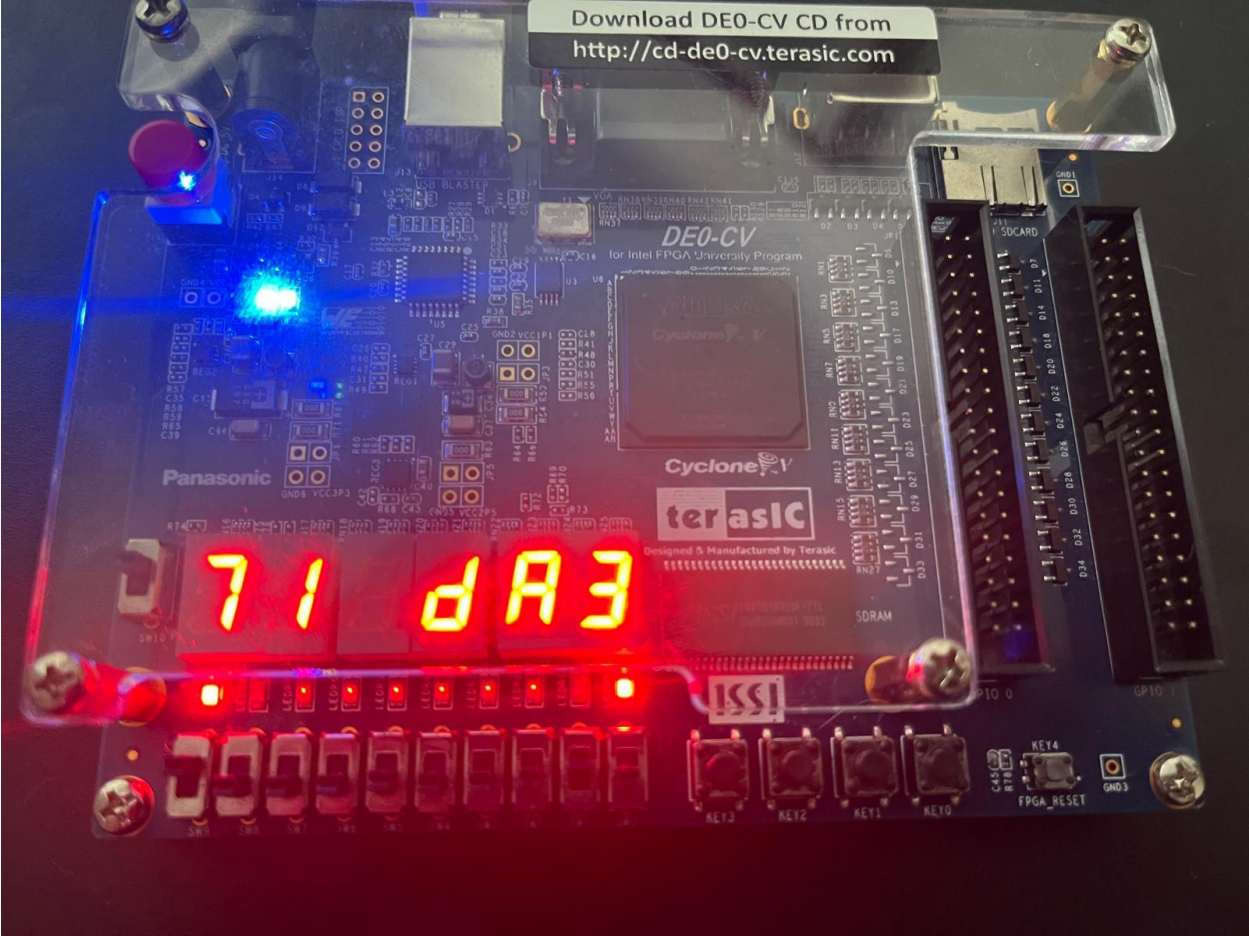
DATA MEMORY

KEY0 KEY1 KEY2 KEY3

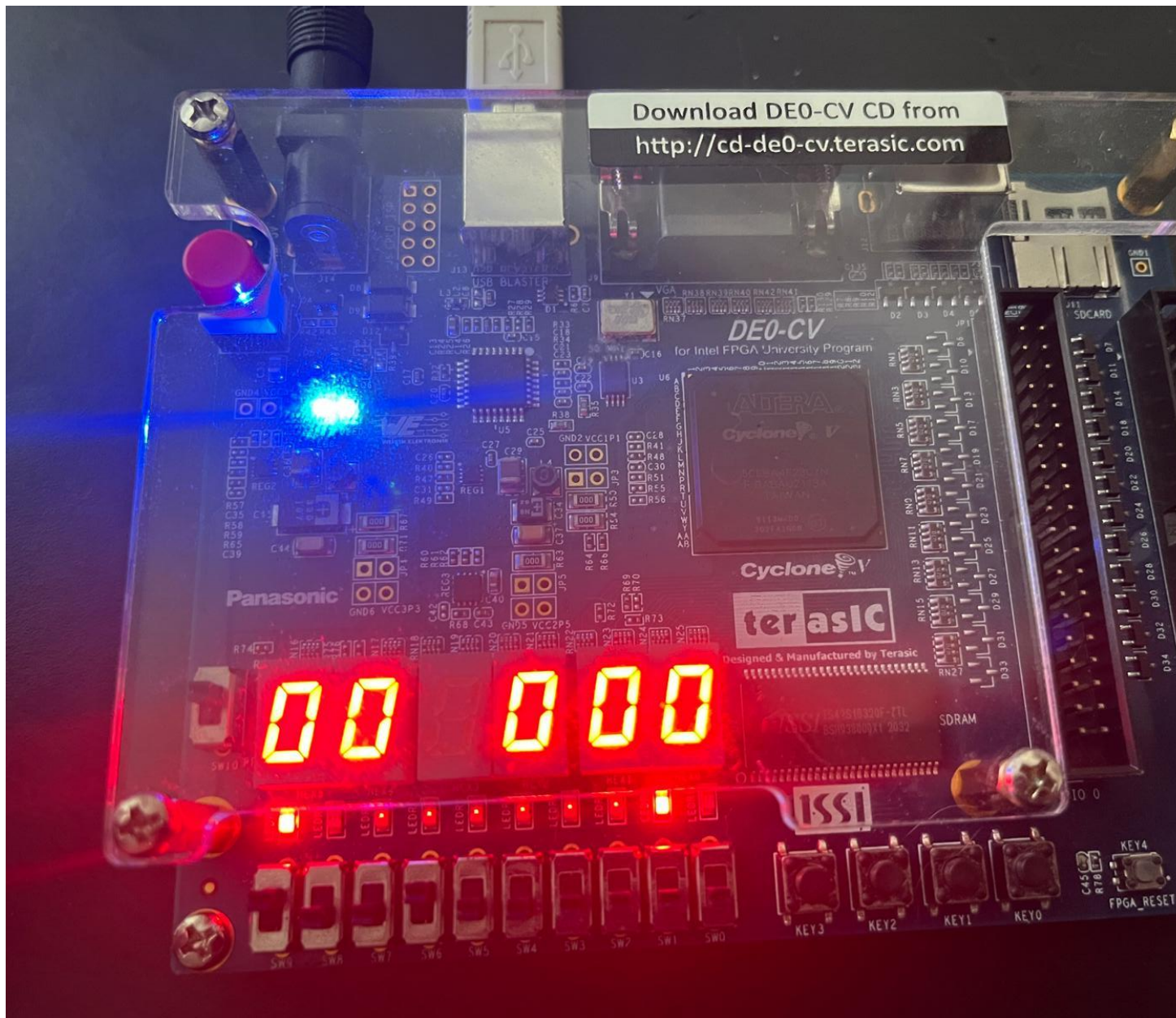
KEY4

FPGA_RESET

GND3



Download DE0-CV CD from
<http://cd-de0-cv.terasic.com>



Download DE0-CV CD from
<http://cd-de0-cv.terasic.com>

DE0-CV

for Intel FPGA University Program

ALTERA
Cyclone V

Cyclone V

terasic

Designed & Manufactured by Terasic

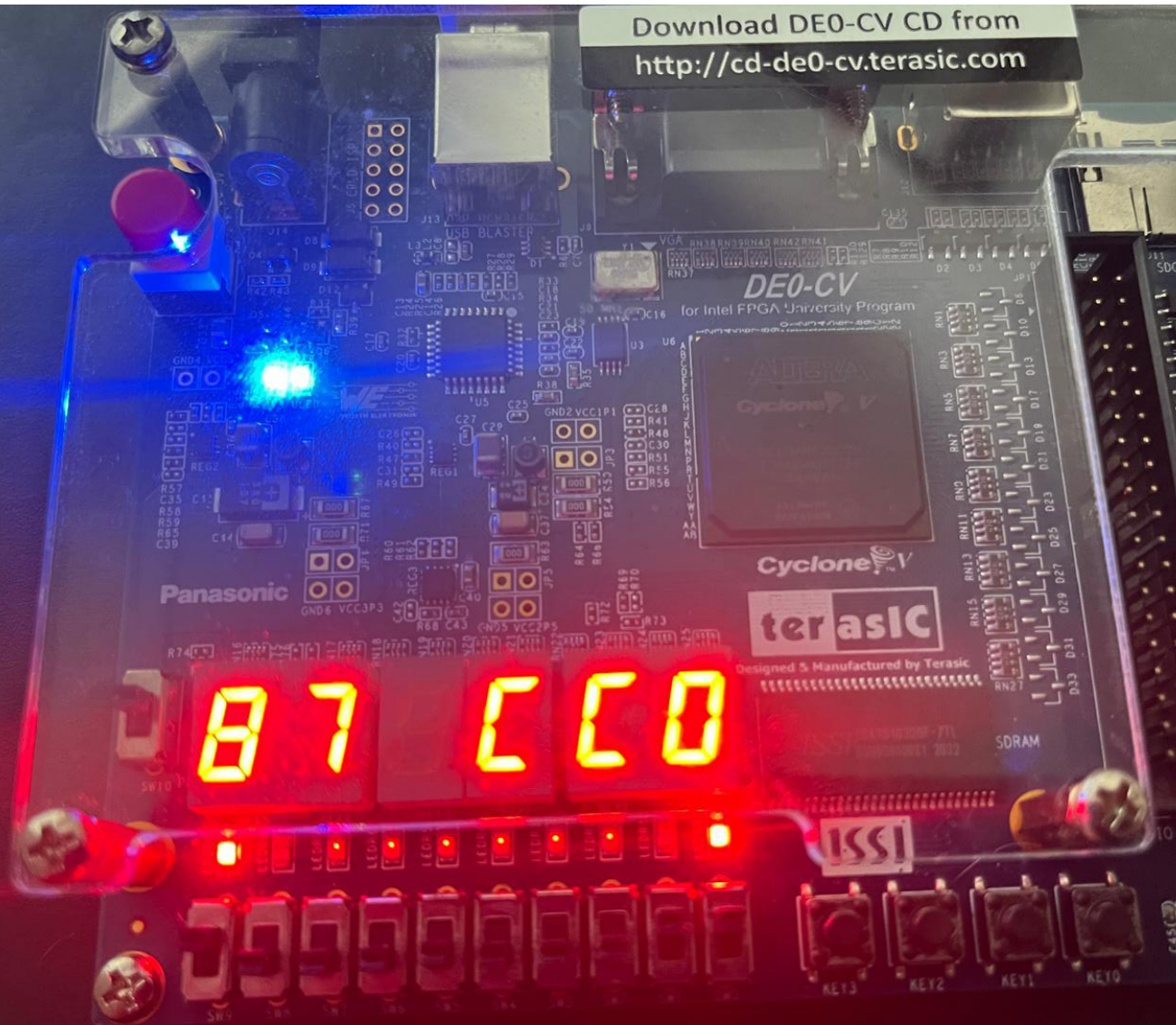
1943516320F-7TL
05091800X1 2002

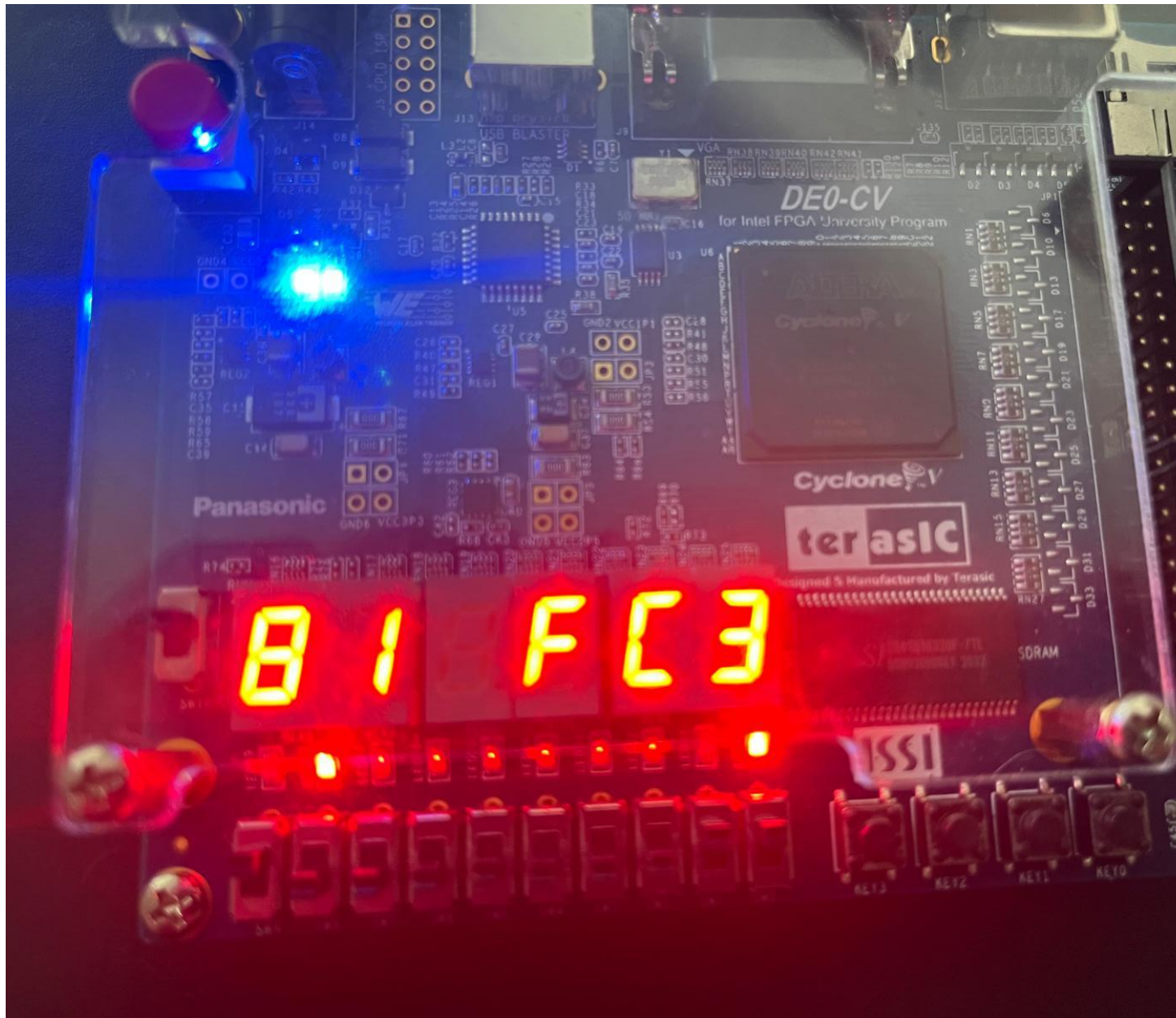
SDRAM

ISSI

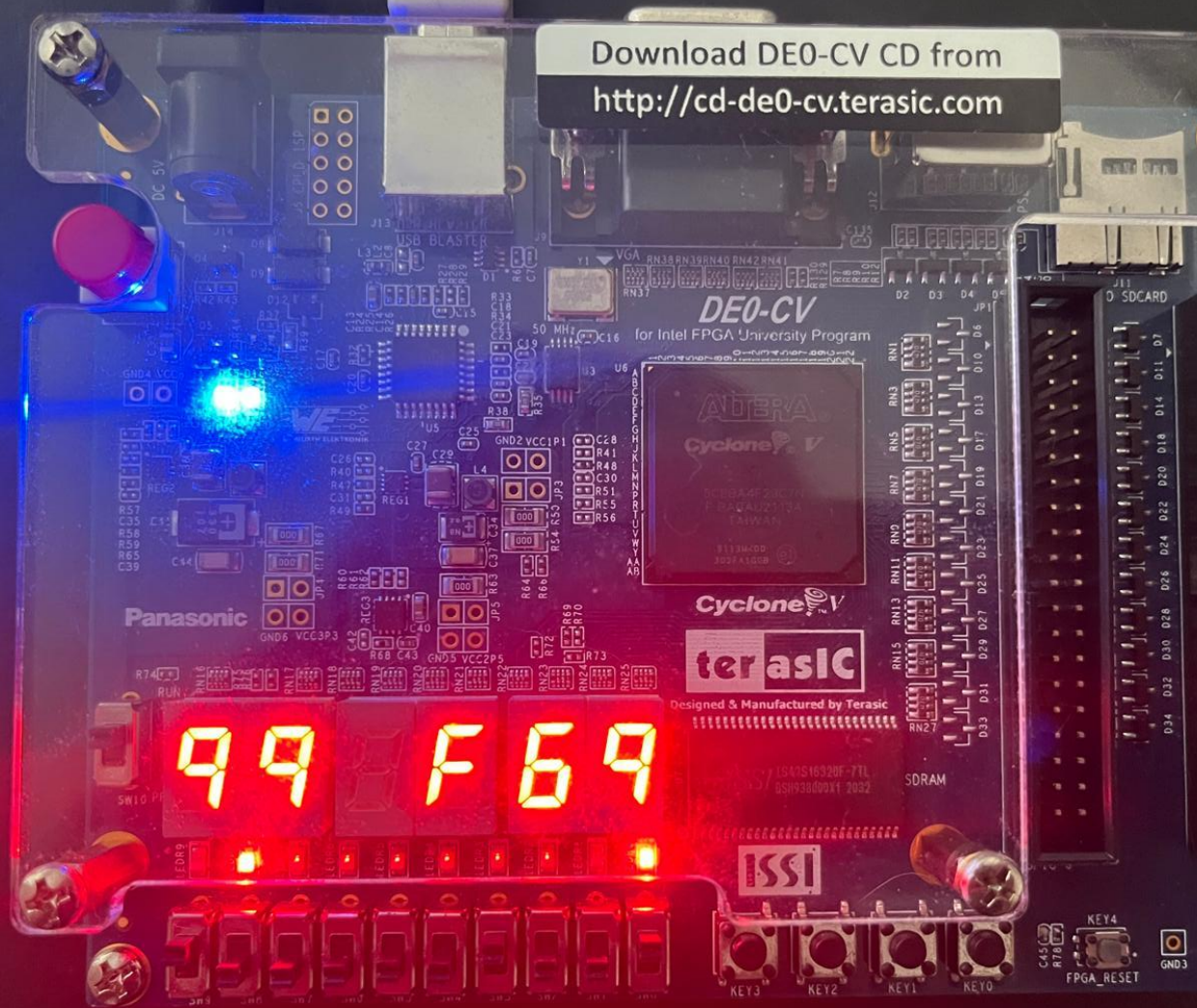
FPGA_RESET

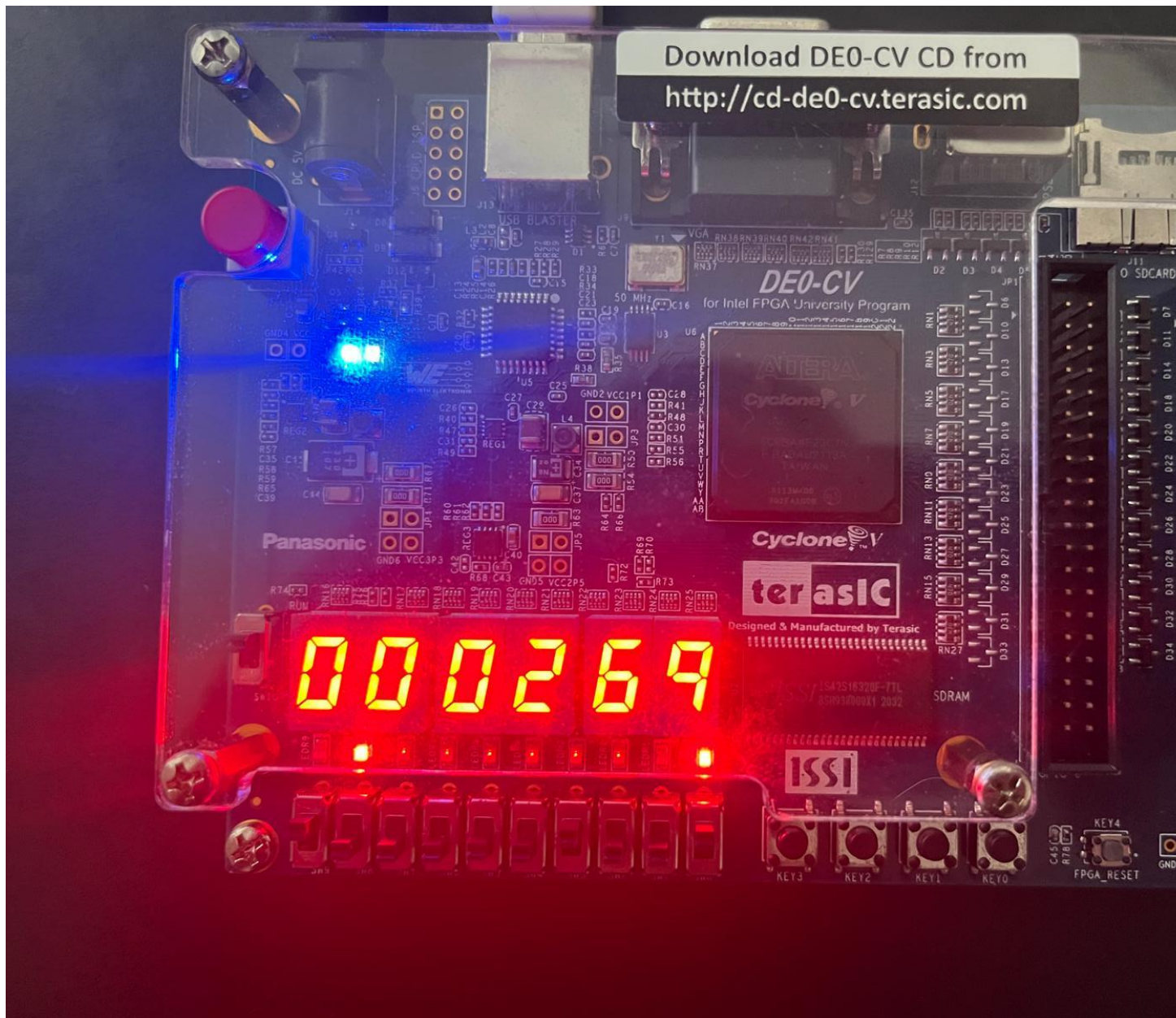
Download DE0-CV CD from
<http://cd-de0-cv.terasic.com>





Download DE0-CV CD from
<http://cd-de0-cv.terasic.com>





Reset is pressed.

Video Demo:

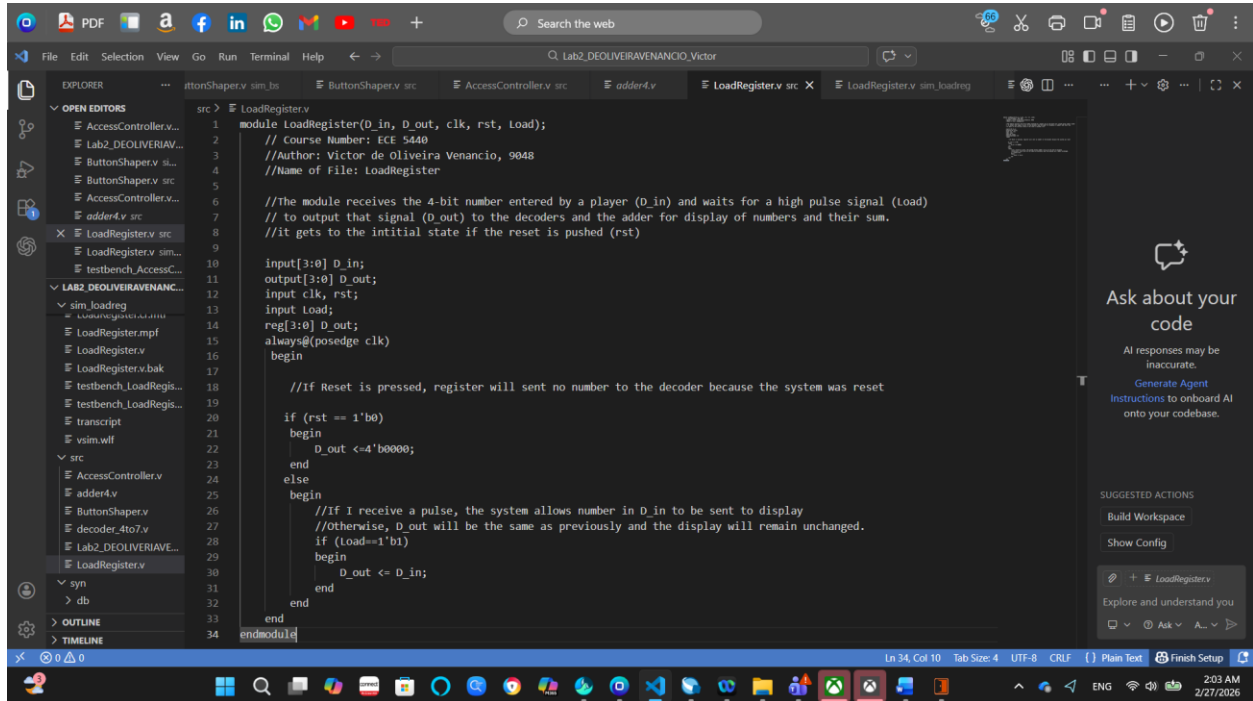
<https://drive.google.com/file/d/1HiRbVAIXNJ48m4y17WinU6Nvm92eGyte/view?usp=sharing>

Conclusion

The Lab was overall successful. The log in process worked as expected and the buttons are working. The FPGA board and the displays behaved as expected and I also included the bonus feature. The leftmost LED was on whenever the central display showed F.

Appendix:

LoadRegister



```
1 module LoadRegister(D_in, D_out, clk, rst, Load);
2     // Course Number: ECE 5440
3     // Author: Victor de Oliveira Venancio, 9048
4     // Name of File: LoadRegister
5
6     //The module receives the 4-bit number entered by a player (D_in) and waits for a high pulse signal (Load)
7     // to output that signal (D_out) to the decoders and the adder for display of numbers and their sum.
8     //it gets to the initial state if the reset is pushed (rst)
9
10    input[3:0] D_in;
11    output[3:0] D_out;
12    input clk, rst;
13    input Load;
14    reg[3:0] D_out;
15    always@(posedge clk)
16    begin
17
18        //If Reset is pressed, register will sent no number to the decoder because the system was reset
19
20        if (rst == 1'b0)
21        begin
22            D_out <= 4'b0000;
23        end
24        else
25        begin
26            //If I receive a pulse, the system allows number in D_in to be sent to display
27            //Otherwise, D_out will be the same as previously and the display will remain unchanged.
28            if (Load==1'b1)
29            begin
30                D_out <= D_in;
31            end
32        end
33    end
34 endmodule
```

ButtonShaper

This screenshot shows the Visual Studio Code editor with the file `ButtonShaper.v` open. The code defines a Verilog module `ButtonShaper` with inputs `B_in`, `B_out`, `clk`, and `rst`. It includes comments about the course (ECE 5440) and author (Victor de Oliveira Venancio, 9048). The module's functionality is described in comments: it takes a button push signal (`B_in`) and converts it to a one high pulse (`B_out`), which is then applied to a register (`reg`) to get the initial state if the reset is pushed (`rst`). The code uses a `case` statement to handle the `State` transitions: `INIT` (waits for the button to be pushed), `PULSE` (sends only one high pulse signal), and `WAIT` (output remains low until the button is released). The status bar at the bottom indicates the cursor is at line 47, column 23, with 4 spaces and UTF-8 encoding.

```
module ButtonShaper(B_in,B_out,clk,rst);
// Course Number: ECE 5440
//Author: Victor de Oliveira Venancio, 9048
//Name of File: ButtonShaper

//The module takes the signal of a button being pushed (B_in) and converts it to one high pulse (B_out)
// so it gets one defined output wherever that button is applied to
//it gets to the initial state if the reset is pushed (rst)

input B_in;
output B_out;
input clk, rst;
reg B_out;
parameter INIT=0, PULSE=1, WAIT=2;
reg[1:0] State, Statenext;

always@(State,B_in)begin
    case(State)
        INIT: begin // waits for the button to be pushed
            B_out=1'b0;
            if(B_in==1'b0)
                Statenext = PULSE;
            else
                Statenext = INIT;
        end
        PULSE: begin
            B_out=1'b1; // sends only one high pulse signal.
            Statenext = WAIT;
        end
        WAIT: begin
            B_out=1'b0;
            if(B_in==1'b1)
                Statenext = INIT;
            else
                Statenext = WAIT;
        end
    endcase
end
```

This screenshot shows the Visual Studio Code editor with the file `ButtonShaper.v` open. The code is the same as the previous screenshot, but it now includes the `AccessController.v` module. The `AccessController.v` module is defined as a `module` with inputs `B_in`, `B_out`, `clk`, and `rst`. It includes comments about the course (ECE 5440) and author (Victor de Oliveira Venancio, 9048). The module's functionality is described in comments: it takes a button push signal (`B_in`) and converts it to a one high pulse (`B_out`), which is then applied to a register (`reg`) to get the initial state if the reset is pushed (`rst`). The code uses a `case` statement to handle the `State` transitions: `INIT` (waits for the button to be pushed), `PULSE` (sends only one high pulse signal), and `WAIT` (output remains low until the button is released). The status bar at the bottom indicates the cursor is at line 47, column 23, with 4 spaces and UTF-8 encoding.

```
module ButtonShaper(B_in,B_out,clk,rst);
// Course Number: ECE 5440
//Author: Victor de Oliveira Venancio, 9048
//Name of File: ButtonShaper

//The module takes the signal of a button being pushed (B_in) and converts it to one high pulse (B_out)
// so it gets one defined output wherever that button is applied to
//it gets to the initial state if the reset is pushed (rst)

input B_in;
output B_out;
input clk, rst;
reg B_out;
parameter INIT=0, PULSE=1, WAIT=2;
reg[1:0] State, Statenext;

always@(State,B_in)begin
    case(State)
        INIT: begin // waits for the button to be pushed
            B_out=1'b0;
            if(B_in==1'b0)
                Statenext = PULSE;
            else
                Statenext = INIT;
        end
        PULSE: begin
            B_out=1'b1; // sends only one high pulse signal.
            Statenext = WAIT;
        end
        WAIT: begin
            B_out=1'b0;
            if(B_in==1'b1)
                Statenext = INIT;
            else
                Statenext = WAIT;
        end
    endcase
end

always@(posedge clk) begin
    if(rst==1'b0)
        State <= WAIT;
    else
        State <= Statenext;
    end
endmodule
```

AccessController

```

module AccessController(Password_digit, Third_button, clk, rst, RNG_Gen_in, Load_P_in, RNG_Gen_out, Load_P_out, Logged_in, Logged_out, time_reconfig, time_enable, time_out);
// Course Number: ECE 5448
//Author: Victor de Oliveira Venancio, 9048
//Name of File: AccessController

//The module restricts access to users that are logged in when they enter the password that consists of the woner's 4- last digits
// each digit is entered in binary form using the 4 leftmost switches in FPGA (Third_button) and pressing the password button to
//enter each digit (Third_button)
//if the authentication is successful it turns on a LED that shows users are logged in (Logged_in)
//otherwise it keeps on the LED that should they are not logged in (Logged_out)
// It also has the bonus feature of pressing the password button while logged in to log out
//It gets to the intitial state if the reset is pushed (rst)
//if log in is successful it sets a timer to 99 with time_reconfig,
//It waits for player to pree game start button (Third_button=1) to enable game play (time_enable=1)
// blocks all buttons if time is over
// if restarted game (Third_button=1) then it recofigures timer and waits for new game to start
//Comment: the correct password to log in is 9048

input[3:0] Password_digit;
output RNG_Gen_out, Load_P_out, Logged_in, Logged_out, time_reconfig, time_enable;
input clk, rst;
input Third_button, RNG_Gen_in, Load_P_in, time_out;
reg RNG_Gen_out, Load_P_out, Logged_in, Logged_out, Flag, time_reconfig, time_enable;
parameter DIGIT1=0, DIGIT2=1, DIGIT3=2, DIGIT4=3, VERIFY=4, PASSED=5, WAITGAME=6, GAMEPLAY=7, GAMEOVER=8;
reg[3:0] State;
always@(posedge clk) begin
    if(rst==1'b0)begin
        RNG_Gen_out <= 1'b1;
        Load_P_out <=1'b0;
        Logged_in <=1'b0;
        Logged_out <=1'b1;
        time_reconfig <=1'b0;
        time_enable <=1'b0;
        State <= DIGIT1;
    end
    else begin
        case(State)
            DIGIT1: begin //It begins as a log out code
                Flag <=1'b1; //It set flag high as another chance for the user to set the password correctly
                RNG_Gen_out <= 1'b1;
                Load_P_out <=1'b0;
                Logged_in <=1'b0;
                Logged_out <=1'b1;
                time_reconfig <=1'b0;
                time_enable <=1'b0;
                if(Third_button==1'b1)begin
                    if(Password_digit==4'b1001) begin // correct digit is 9
                        State <=DIGIT2;
                    end
                    else begin
                        Flag <=1'b0; // Password will be incorrect because this is not the right digit
                        State <=DIGIT2;
                    end
                end
            end
            else begin
                State <=DIGIT1; // it waits for a digit to be registered by Third_button
            end
        end

        DIGIT2: begin
            RNG_Gen_out <= 1'b1;
            Load_P_out <=1'b0;
            Logged_in <=1'b0;
            Logged_out <=1'b1;
            time_reconfig <=1'b0;
            time_enable <=1'b0;
        end
    end
end

```

```

module AccessController(Password_digit, Third_button, Clk, rst, RNG_Gen_in, Load_P_in, RNG_Gen_out, Load_P_out, Logged_in, Logged_out, time_reconfig, time_enable, time_out);
always@(posedge clk) begin
    else begin
        case(State)

            DIGIT2: begin
                RNG_Gen_out <= 1'b1;
                Load_P_out <= 1'b0;
                Logged_in <= 1'b0;
                Logged_out <= 1'b1;
                time_reconfig <= 1'b0;
                time_enable <= 1'b0;
                if(Third_button==1'b1)begin
                    if(Password_digit==4'b0000) begin // correct digit is 0
                        State <=DIGIT3;
                    end
                    else begin
                        Flag <=1'b0; // Password will be incorrect because this is not the right digit
                        State <=DIGIT3;
                    end
                end
            end
            else begin
                State <=DIGIT2; // it waits for a digit to be registered by Third_button
            end
        end

        DIGIT3: begin
            RNG_Gen_out <= 1'b1;
            Load_P_out <= 1'b0;
            Logged_in <= 1'b0;
            Logged_out <= 1'b1;
            time_reconfig <= 1'b0;
            time_enable <= 1'b0;
            if(Third_button==1'b1)begin
                if(Password_digit==4'b0100) begin // correct digit is 4
                    State <=DIGIT4;
                end
                else begin
                    Flag <=1'b0; // Password will be incorrect because this is not the right digit
                    State <=DIGIT4;
                end
            end
            else begin
                State <=DIGIT3; // it waits for a digit to be registered by Third_button
            end
        end

        DIGIT4: begin
            RNG_Gen_out <= 1'b1;
            Load_P_out <= 1'b0;
            Logged_in <= 1'b0;
            Logged_out <= 1'b1;
            time_reconfig <= 1'b0;
            time_enable <= 1'b0;
            if(Third_button==1'b1)begin
                if(Password_digit==4'b1000) begin // correct digit is 8
                    State <=VERIFY;
                end
                else begin
                    Flag <=1'b0; // Password will be incorrect because this is not the right digit
                    State <=VERIFY;
                end
            end
        end
    end
end

```

```

module AccessController(Password_digit, Third_button, clk, rst, RNG_Gen_in, Load_P_in, RNG_Gen_out, Load_P_out, Logged_in, Logged_out, time_reconfig, time_enable, time_out);
always@(posedge clk) begin
    else begin
        case(State)
            enu
            end

            DIGIT4: begin
                RNG_Gen_out <= 1'b1;
                Load_P_out <= 1'b0;
                Logged_in <= 1'b0;
                Logged_out <= 1'b1;
                time_reconfig <= 1'b0;
                time_enable <= 1'b0;
                if(Third_button==1'b1)begin
                    if(Password_digit==4'b1000) begin // correct digit is 8
                        State <= VERIFY;
                    end
                    else begin
                        Flag <= 1'b0; // Password will be incorrect because this is not the right digit
                        State <= VERIFY;
                    end
                end
            end
            else begin
                State <= DIGIT4; // it waits for a digit to be registered by Third_button
            end
        end

        VERIFY: begin
            RNG_Gen_out <= 1'b1;
            Load_P_out <= 1'b0;
            Logged_in <= 1'b0;
            Logged_out <= 1'b1;
            time_reconfig <= 1'b0;
            time_enable <= 1'b0;
            if(Flag == 1'b1) begin // Password is correct
                State <= PASSED;
            end
            else begin // at least one digit is wrong
                State <= DIGIT1; //let user try again
            end
        end

        PASSED: begin
            RNG_Gen_out <= 1'b1;
            Load_P_out <= 1'b0;
            Logged_in <= 1'b1;
            Logged_out <= 1'b0;
            Flag <= 1'b1;
            time_reconfig <= 1'b1;
            time_enable <= 1'b0;
            State <= WAITGAME;
        end

        WAITGAME: begin
            RNG_Gen_out <= 1'b1;
            Load_P_out <= 1'b0;
            Logged_in <= 1'b1;
            Logged_out <= 1'b0;
            Flag <= 1'b1;
            time_reconfig <= 1'b0;
            time_enable <= 1'b0;
            if(Third_button==1'b1) begin

```

```

module AccessController(Password_digit, Third_button, Clk, rst, RNG_Gen_in, Load_P_in, RNG_Gen_out, Load_P_out, Logged_in
always@(posedge clk) begin
    else begin
        case(State)
            PASSED: begin
                State <= WAITGAME;
            end

            WAITGAME: begin
                RNG_Gen_out <= 1'b1;
                Load_P_out <= 1'b0;
                Logged_in <= 1'b1;
                Logged_out <= 1'b0;
                Flag <= 1'b1;
                time_reconfig <= 1'b0;
                time_enable <= 1'b0;
                if(Third_button==1'b1) begin
                    State <= GAMEPLAY;
                end
                else begin // at least one digit is wrong
                    State <= WAITGAME;
                end
            end

            GAMEPLAY: begin
                RNG_Gen_out <= RNG_Gen_in;
                Load_P_out <= Load_P_in;
                Logged_in <= 1'b1;
                Logged_out <= 1'b0;
                Flag <= 1'b1;
                time_reconfig <= 1'b0;
                time_enable <= 1'b1;
                if(time_out==1'b1) begin
                    State <= GAMEOVER;
                end
                else begin
                    State <= GAMEPLAY;
                end
            end

            GAMEOVER: begin
                RNG_Gen_out <= 1'b1;
                Load_P_out <= 1'b0;
                Logged_in <= 1'b1;
                Logged_out <= 1'b0;
                Flag <= 1'b1;
                time_reconfig <= 1'b0;
                time_enable <= 1'b0;
                if(Third_button==1'b1) begin
                    State <= PASSED;
                end
                else begin
                    State <= GAMEOVER;
                end
            end

            default: begin
                Flag <= 1'b1; //it set flag high as another chance for the user to set the password correctly
                RNG_Gen_out <= 1'b1;
                Load_P_out <= 1'b0;
                Logged_in <= 1'b0;
            end
        endcase
    end
end

```

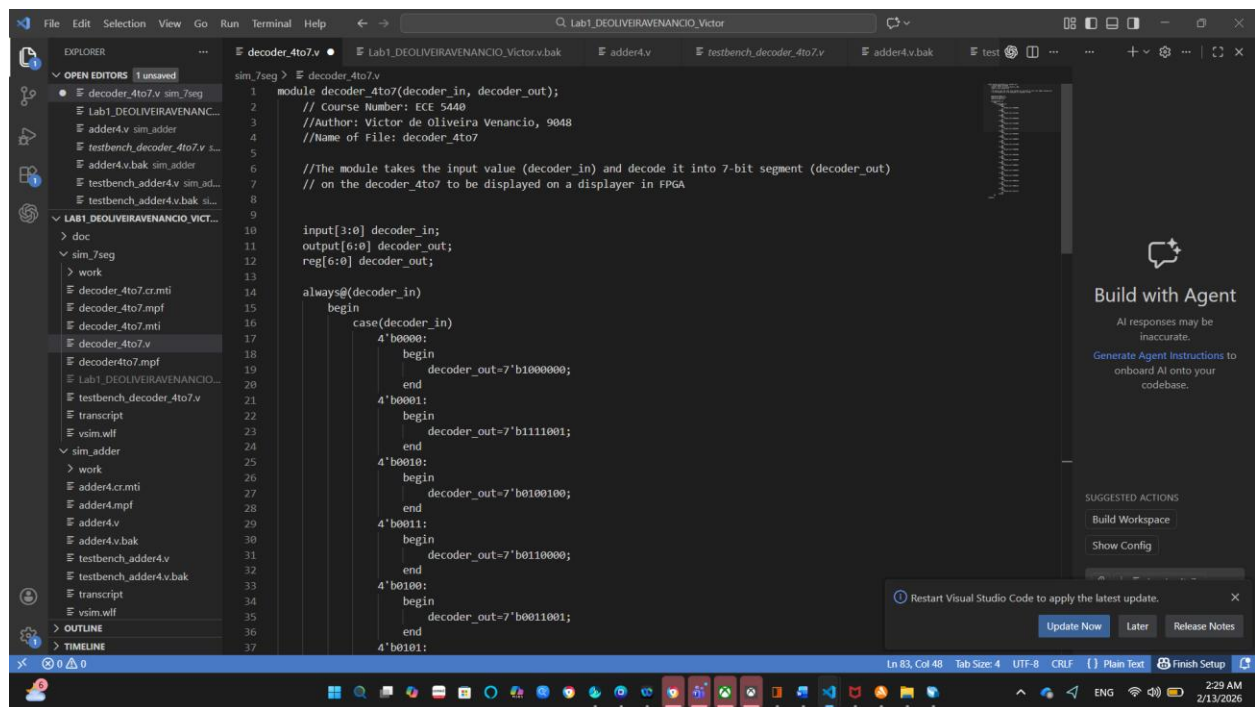


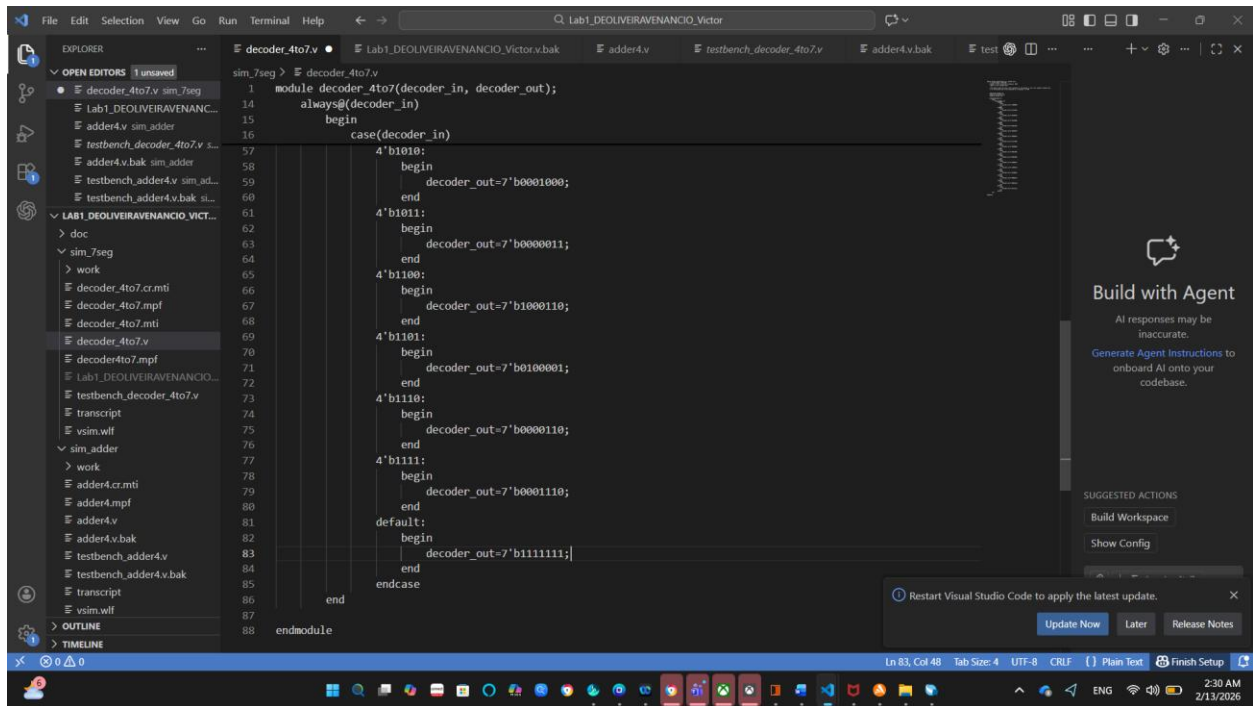
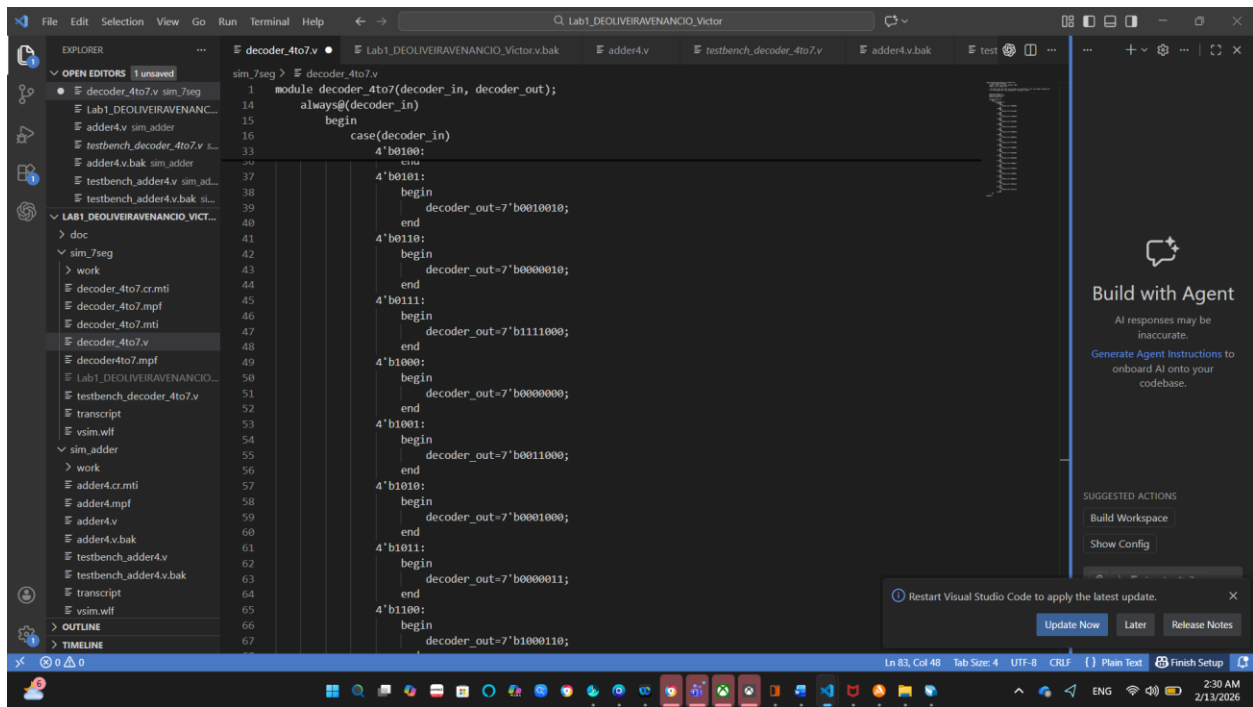
```

1 module AccessController(Passwd_digit, Third_button, clk, rst, RNG_Gen_in, Load_P_in, RNG_Gen_out, Load_P_out, Logged_in, Logged_
2 always@(posedge clk) begin
3     else begin
4         case(State)
5             GAMEOVER: begin
6                 // State of GAMEOVER,
7                 // ...
8                 end
9             end
10        end
11    end
12
13    default: begin
14        Flag <= 1'b1; //it set flag high as another chance for the user to set the password correctly
15        RNG_Gen_out <= 1'b1;
16        Load_P_out <= 1'b0;
17        Logged_in <= 1'b0;
18        Logged_out <= 1'b1;
19        time_reconfig <= 1'b0;
20        time_enable <= 1'b0;
21        State <= DIGIT1;
22    end
23 endcase
24 end
25 end
26 endmodule

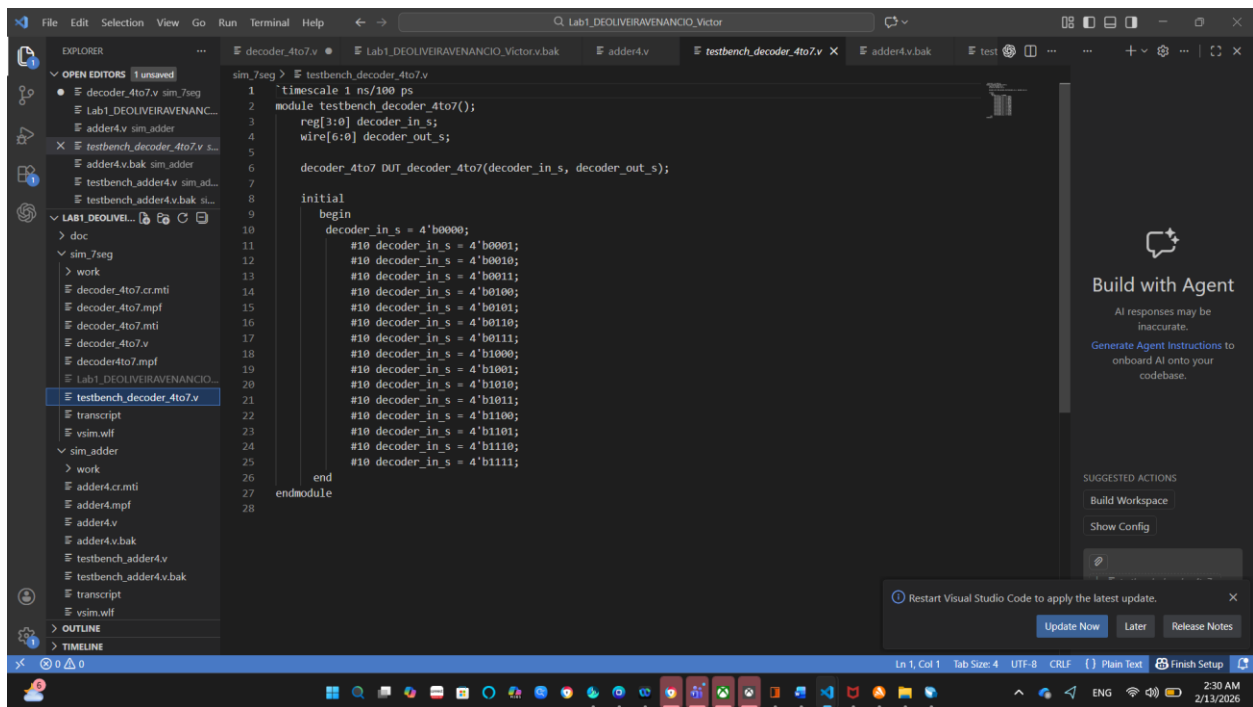
```

Decoder_4to7

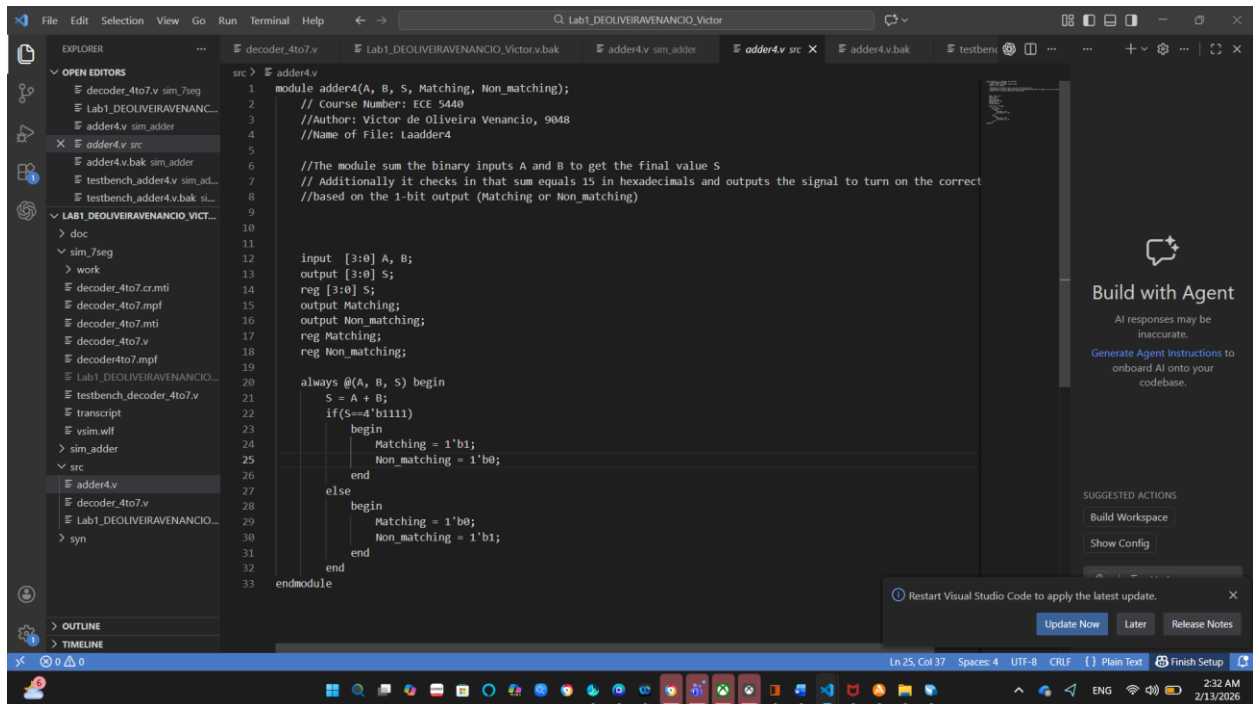




Testbench for decoder_4to7



Adder4



BCD Counter

```

C:\AppDemos\Lab3_DEOLIVEIRAVENANCIO_Victor\src\BCD_Counter.v D_Counter.v
1 module BCD_Counter(RNG_Number, P_number, time_reconfig, Load, clk, rst, score_ones, score_tens);
2
3 // Course Number: ECE 5440
4 // Author: Victor de Oliveira Venancio, 9048
5 // Name of File: BCD_Counter
6 //
7 // This module counts the total number of successful rounds in the game.
8 // It receives the random number from RNG in RNG_Number and the player's number directly from player's switches.
9 // When it receives a load, if the sum between RNG_Number and P_number is 15 it sums 1 to the score defined by score_ones (ones) and score_tens (tens)
10 // If time_reconfig is high, the score is set back to 0 preparing the system for a new game.
11 // If rst is low, the whole score is also reset to 0.
12
13 input Load, time_reconfig, clk, rst;
14 input [3:0] RNG_Number, P_number;
15 output [3:0] score_ones, score_tens;
16 reg [3:0] score_ones, score_tens;
17
18 always @(posedge clk) begin
19     if (rst == 1'b0) begin
20         score_ones <= 4'd0;
21         score_tens <= 4'd0;
22     end
23     else if (time_reconfig == 1'b1) begin
24         score_ones <= 4'd0;
25         score_tens <= 4'd0;
26     end
27     else begin
28         if (Load == 1'b1) begin
29             if ((RNG_Number + P_number) == 4'd15) begin
30                 if (score_ones < 4'd9) begin
31                     score_ones <= score_ones + 4'd1;
32                 end
33                 else begin
34                     if (score_tens < 4'd9) begin
35                         score_tens <= score_tens + 4'd1;
36                         score_ones <= 4'd0;
37                     end
38                     else begin
39                         score_ones <= 4'd9;
40                         score_tens <= 4'd9;
41                     end
42                 end
43             end
44         end
45     end
46 end
47 endmodule

```

Multiplexer

```

module Multiplexer(sum, timeOut, score_ones, score_tens, disp_right, disp_left);
    // Course Number: ECE 5440
    // Author: Victor de Oliveira Venancio, 9048
    // Name of File: ScoreSumMux
    //
    // This module selects whether the middle displays show
    // the game sum or the final score based on a switch (Score_mode)
    // when timeOut is high, it displays the total score, otherwise it displays the sum
    // Gameplay starts by pushing the Game Start button
    // or when logged out (rst)
    // which is when disp_left and disp_right display sum in disp_right (timeOut=0)
    // Gameover starts after time out signal comes from timer equal to 00 (timeOut high)
    // which is when displays show total score in deicmals
    // (disp_left shows ones portion and disp_right shows tens portion) (timeOut=1)
    // Because the left display (disp_left) should be off during sum output
    // it is connected to a different version of the decoder_4to7 that truns off for input greater than 9
    // that way, for the case to show the sum, a value greater than 9 is assigned to disp_left
    // to turn its corresponding display off

    input [3:0] sum, score_ones, score_tens;
    input timeOut;

    output [3:0] disp_right, disp_left;

    reg [3:0] disp_right, disp_left;

    // Display selection
    always @(sum or score_ones or score_tens or timeOut) begin
        if (timeOut == 1'b1) begin
            disp_left  = score_tens;
            disp_right = score_ones;
        end
        else begin
            disp_left  = 4'd15; // Connected to a decoder_4to7_OnOff that will turn display off
            disp_right = sum;
        end
    end

endmodule

```

Two Digit Timer


```

rc > E twoDigitTimer.v
1 module twoDigitTimer(clk, rst, time_enable_s, time_reconfig_s, time_out_s, tens_digit, ones_digit);
2     // Course Number: ECE 5440
3     //Author: Victor de Oliveira Venancio, 9048
4     //Name of File: TwoDigitTimer
5
6     //This is the two digit timer that will receive the time_reconfig from access controller to be set to 99
7     //After game start button is pushed time_enable_s becomes high and timer starts to decrease from the
8     //1second_timer from counttms_s, countTo100_s and countTo10_s then it makes the onesDigitTimer decrease its digit by 1 each second by transmitting
9     // a high pulse through s1Timeout_s. When onesDigitTimer outputs 0 it borrows from tensDigitTimer if it is possible and makes the tensDigitTimer decrease its digit by 1
10    // if it is not possible, the NBD_1 will output a high pulse signal for AccessController to block buttons turn time_enable off and end the game
11
12    input clk, rst, time_reconfig_s, time_enable_s;
13    output time_out_s;
14    output [3:0] tens_digit, ones_digit;
15
16    wire ms1Timeout_s, ms100Timeout_s, s1Timeout_s, BD_1, NBD_1, BU_1, NBU_1, BD_10, NBD_10, BU_10;
17
18    counttms counttms_s(time_enable_s, clk, rst, ms1Timeout_s);
19    countTo100 countTo100_s(time_enable_s, ms1Timeout_s, clk, rst, ms100Timeout_s);
20    countTo10 countTo10_s(time_enable_s, ms100Timeout_s, clk, rst, s1Timeout_s);
21
22    assign BD_1 = s1Timeout_s;
23    assign BD_10 = BU_1;
24    assign NBU_1 = NBD_10;
25    assign time_out_s = NBD_1;
26
27    digitTimer onesDigitTimer(BD_1, NBD_1, BU_1, NBU_1, time_reconfig_s, clk, rst, ones_digit);
28    digitTimer tensDigitTimer(BD_10, NBD_10, BU_10, 1'b1, time_reconfig_s, clk, rst, tens_digit);
29
30 endmodule

```

RNG

```

1 module RNG(RNG_Gen,clk, rst,Random_num);
2     // Course Number: ECE 5440
3     //Author: Victor de Oliveira Venancio, 9048
4     //Name of File: RNG
5
6     //The module generates a random number between 0 and 15 by pushing a button when player is logged in and the game is on
7     //it inverts the low RNG_Gen signal from button push (active low) and wires the resulting signal to an RNG RNG_Counter
8     //it will change the count_out output while button is pushed by incrementing 1 from 0 to 15, then restart the cycle until button is released
9     //after button is released, the count_out output will remain in its latest value
10
11    input RNG_Gen, clk, rst;
12    output [3:0] Random_num;
13
14    wire count_s;
15    wire [3:0] count_out_s;
16
17    assign count_s = ~RNG_Gen;
18    assign Random_num = count_out_s;
19
20    RNG_Counter RNGCounter(clk, rst, count_s, count_out_s);
21
22 endmodule

```